CHAPTER-7
ENCODERS, DECODERS, MULTIPLEXERS, AND DEMULTIPLEXERS

7.1 Introduction to Encoders

An encoder is a logic circuit that accepts one of many inputs at a given time and generates a code corresponding to that input. The process of converting an input to an output code is called encoding. Fig.1 shows a general block diagram of an encoder.

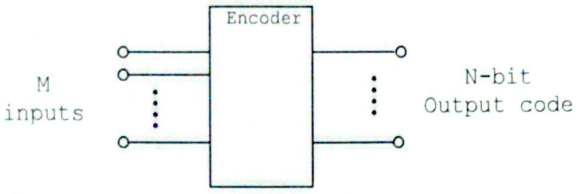


Fig.1: A general block diagram of an encoder

The encoder has M input lines but only one input is activated at a time and produces an N-bit output code. The relation between input and output lines is $2^N \geq M$. For example, the decimal-to-BCD encoder has 10 input and 4 output lines; the octal-to-binary encoder has 8 input and 3 output lines.

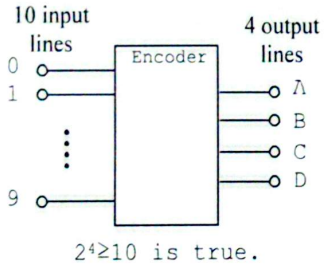


Fig.2: Decimal-to-BCD encoder

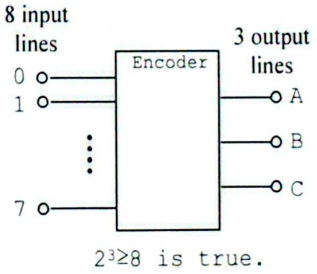


Fig.3: Octal-to-Binary encoder

7.1.1 Decimal-to-BCD Encoder

The Decimal-to-BCD encoder has 10 inputs (0 through 9), but only one of them is activated at a time and produces a 4-bit BCD code corresponding to that input. This is

also known as a 10-line-to-4-line encoder. The block diagram is shown in Fig.2. Its truth table is given below.

Truth table of Decimal-to-BCD encoder

Decimal Digit Input	Output (BCD Code)			
	A	B	C	D
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

According to the above truth table, if 1 is activated, the encoder will generate the output BCD code 0001. Similarly, if 5 is activated, its output will be 0101. Thus, we can write the expressions for A, B, C, and D as follows:

i. A will be HIGH if 8 or 9 is activated. That is, A=8 or 9 (i.e. A=8+9).
ii. B will be HIGH if 4 or 5 or 6 or 7 is activated. That is, B=4 or 5 or 6 or 7 (i.e. B=4+5+6+7).
iii. C will be HIGH if 2 or 3 or 6 or 7 is activated. That is, C=2 or 3 or 6 or 7 (i.e. C=2+3+6+7).
iv. D will be HIGH if 1 or 3 or 5 or 7 or 9 is activated. That is, D=1 or 3 or 5 or 7 or 9 (i.e. D=1+3+5+7+9).

The logic circuit of the decimal-to-BCD encoder is implemented in Fig.4.

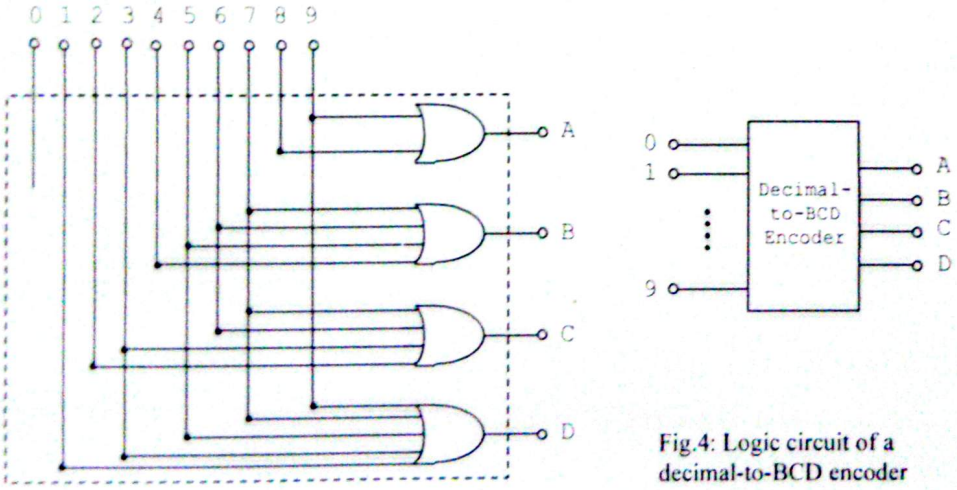


Fig.4: Logic circuit of a decimal-to-BCD encoder

7.1.2 Octal-to-Binary Encoder

The Octal-to-Binary encoder has 8 inputs (0 through 7) but only one of them is activated at a time and produces a 3-bit binary code corresponding to that input. This is also known as a 8-line-to-3-line encoder. The block diagram is shown in Fig.3. Its truth table is given below.

Truth table of Octal-to-Binary encoder

Octal Digit Input	Output (Binary Code)		
	A	B	C
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

According to the above truth table, we can write the output expressions for A, B, and C as follows:

i. A will be HIGH if 4 or 5 or 6 or 7 is activated. So, the logic expression of A is A=4+5+6+7.
ii. B will be HIGH if 2 or 3 or 6 or 7 is activated. So, its expression is B=2+3+6+7.
iii. C will be HIGH if 1 or 3 or 5 or 7 is activated. So, its expression is C=1+3+5+7.

The logic circuit of the octal-to-binary encoder is implemented in Fig.5.

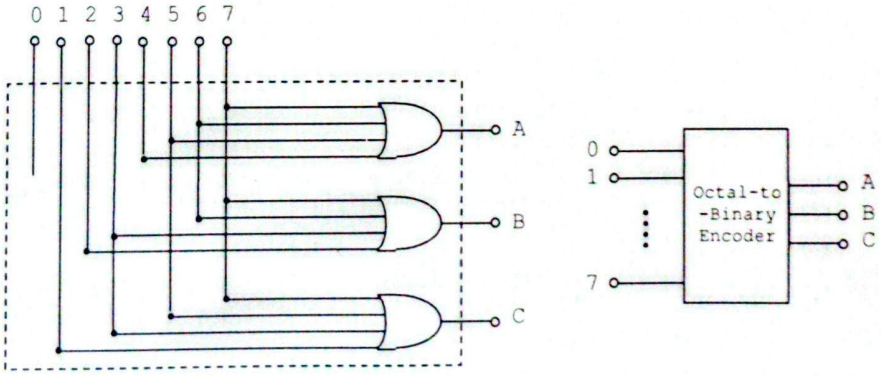


Fig.5: Logic circuit of Octal-to-Binary encoder

When an octal digit is activated (i.e. HIGH), the corresponding 3-bit binary code is produced at the output. For instance, if octal digit 5 is activated (i.e. HIGH), the code A=1, B=0, and C=1 will be produced.

A major drawback of these simple encoders is that, when more than one input is activated at the same time, the encoder will produce a wrong output code. For example, if the digits 2 and 5 are activated simultaneously, the output code A=1, B=1, and C=1 will be produced (which is correct for digit 7). To overcome this problem, a modified version of this simple encoder is used which is called a priority encoder. When two or more inputs are activated simultaneously, a priority encoder will produce a code corresponding to the highest numbered input.

7.1.3 Decimal-to-BCD Priority Encoder

From the truth table of the simple Decimal-to-BCD encoder, we can write the output expressions for A, B, C, and D for priority encoder as follows:

i. A will be HIGH if digit 8 or 9 is activated. So, A=8+9.

ii. B will be HIGH if digit 4 or 5 or 6 or 7 is activated, but digits 8 and 9 are inactivated. So, $B = (4+5+6+7)\ and\ (\overline{8}.\overline{9}) = (4+5+6+7).\overline{8}\overline{9}$

iii. C will be HIGH if digit 2 or 3 or 6 or 7 is activated and digits 4, 5, 8, and 9 are inactivated. So, $C = (2+3).\overline{4}\overline{5}\overline{8}\overline{9} + (6+7).\overline{8}\overline{9}$

v. D will be HIGH if 1 or 3 or 5 or 7 or 9 is activated and 2, 4, 6, and 8 are inactivated. That is,

D=1 if (1 is HIGH and 2, 4, 6, and 8 are LOW)
Or (3 is HIGH and 4, 6, and 8 are LOW)
Or (5 is HIGH and 6 and 8 are LOW)
Or (7 is HIGH and 8 is LOW)
Or (9 is HIGH).

So, $D = 1.\overline{2}\overline{4}\overline{6}\overline{8} + 3.\overline{4}\overline{6}\overline{8} + 5.\overline{6}\overline{8} + 7.\overline{8} + 9$

The logic circuit of the decimal-to-BCD priority encoder is implemented in Fig.6. If more than one digit is activated simultaneously, this priority encoder will generate the BCD code corresponding to the highest numbered input. For example, if digits 2 and 5 are activated simultaneously, from the logic circuit of Fig.6 we will get the following output BCD code.

$$A = 8+9 = 0+0 = 0$$

$$B = (4+5+6+7).\overline{8}\overline{9} = (0+1+0+0).\overline{0}.\overline{0} = (1).1.1 = 1$$

$$C = (2+3).\overline{4}\overline{5}\overline{8}\overline{9} + (6+7).\overline{8}\overline{9} = (1+0).\overline{0}\overline{1}\overline{0}.\overline{0} + (0+0).\overline{0}.\overline{0} = 1.0 + 0.1 = 0$$

$$D = 1.\overline{2}\overline{4}\overline{6}\overline{8} + 3.\overline{4}\overline{6}\overline{8} + 5.\overline{6}\overline{8} + 7.\overline{8} + 9 = 0.\overline{1}\overline{0}\overline{0}.\overline{0} + 0.\overline{0}\overline{0}.\overline{0} + 1.\overline{0}.\overline{0} + 0.\overline{0} + 0 = 0+0+1+0+0 = 1$$

Therefore, the generated BCD code is ABCD=0101 which is correct for digit 5, thus, digit 2 is ignored.

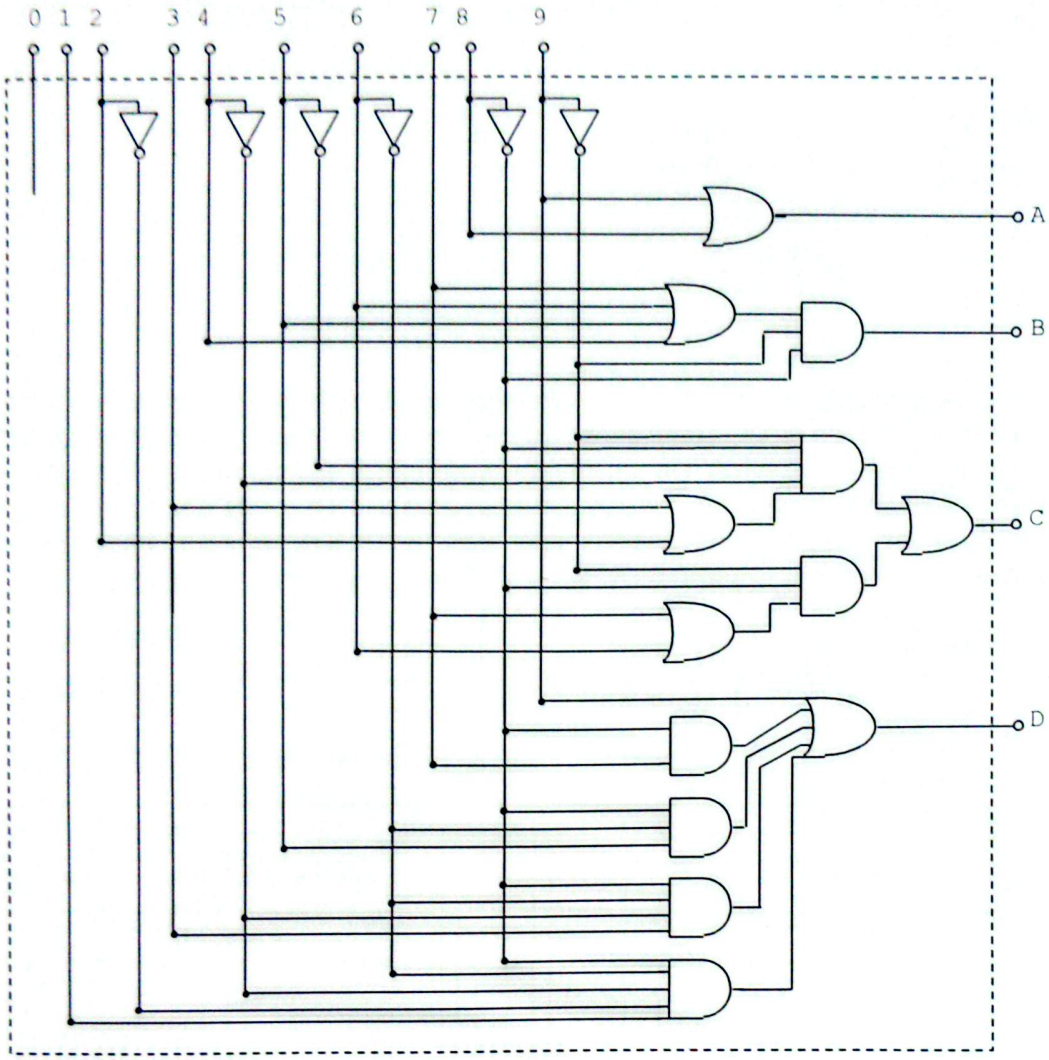


Fig.6: Logic circuit of decimal-to-BCD priority encoder

For handling priority interrupts in computers, microprocessors, and microcontrollers priority encoders are widely be used.

CamScanner

7.2 Introduction to Decoders

A decoder is a logic circuit that accepts a set of inputs (i.e. a code) and activates only one of many outputs corresponding to the code. Actually, it does the reverse function of an encoder. Fig.7 shows a general block diagram of a decoder.

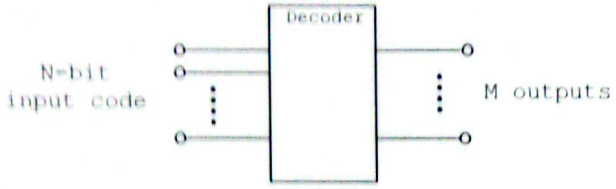


Fig.7: A general block diagram of a decoder

It has an N input lines and M output lines. So, there are 2^N possible input combinations (i.e. codes). For each input combination, only one of the M outputs corresponding to that combination is activated. The relation between inputs and outputs is $2^N \geq M$. For example, a BCD-to-Decimal decoder has 4 input lines and 10 output lines; a Binary-to-Octal decoder has 3 input lines and 8 output lines. In fact, a decoder does the reverse function of an encoder.

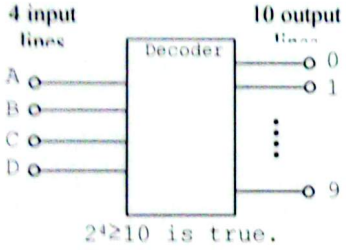


Fig.8: BCD-to-Decimal decoder

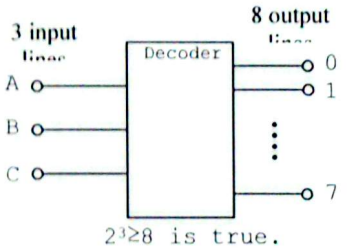


Fig.9: Binary-to-Octal decoder

7.2.1 BCD-to-Decimal Decoder

A BCD-to-Decimal decoder has four input lines (i.e. 4-bit input code) and ten output lines. When a 4-bit input code is applied to the decoder, one of the 10 output lines corresponding to that input code is active. This decoder is also referred to as a 4-line-to-10-line decoder or 1-of-10 decoder. Its truth table is given below.

Truth table

BCD input				Decimal output									
A	B	C	D	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8	D_9
0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	0	1	0	0	0	0	0	0
0	1	0	0	0	0	0	0	1	0	0	0	0	0
0	1	0	1	0	0	0	0	0	1	0	0	0	0
0	1	1	0	0	0	0	0	0	0	1	0	0	0
0	1	1	1	0	0	0	0	0	0	0	1	0	0
1	0	0	0	0	0	0	0	0	0	0	0	1	0
1	0	0	1	0	0	0	0	0	0	0	0	0	1

From the truth table, the expressions for the outputs (for D_0 through D_9) can be written as following:

1. If the BCD input is A=B=C=D=0 (LOW), Output D_0 will be activated. So, the output expression for D_0 is $D_0 = \overline{A}\overline{B}\overline{C}\overline{D}$. Similarly,

2. $D_1 = \overline{A}\overline{B}\overline{C}D$. 3. $D_2 = \overline{A}\overline{B}C\overline{D}$. 4. $D_3 = \overline{A}\overline{B}CD$ 5. $D_4 = \overline{A}B\overline{C}\overline{D}$

6. $D_5 = \overline{A}B\overline{C}D$ 7. $D_6 = \overline{A}BC\overline{D}$. 8. $D_7 = \overline{A}BCD$ 9. $D_8 = A\overline{B}\overline{C}\overline{D}$

10. $D_9 = A\overline{B}\overline{C}D$.

The logic circuit is implemented in Fig.10.

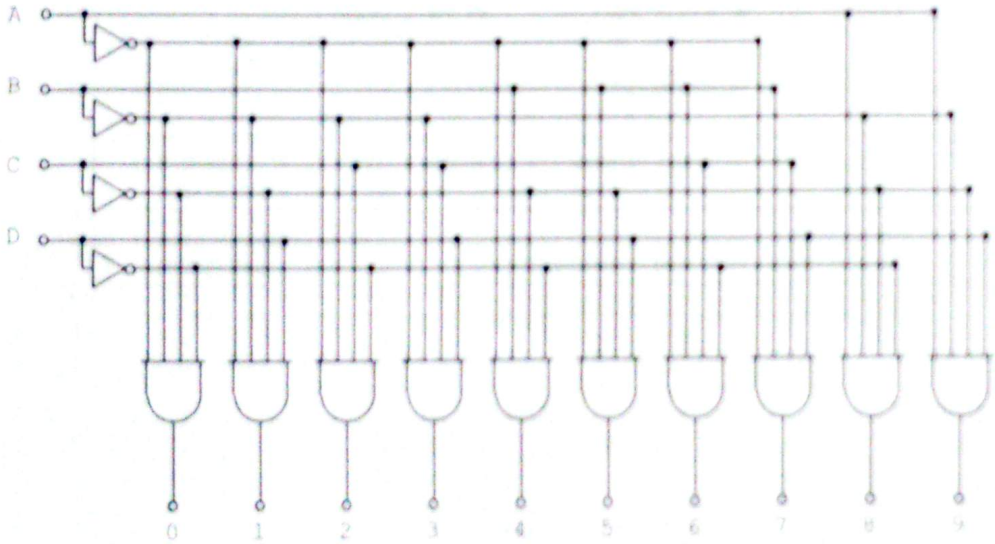


Fig.10. BCD-to-Decimal decoder

7.2.2 Binary-to-Octal Decoder

A Binary-to-Octal decoder has three input lines (i.e. 3-bit input code) and eight output lines. When a 3-bit input code is applied to the decoder, one of the 8 output lines corresponding to that input code is activated. This decoder is also called a 3-line-to-8-line decoder or 1-of-8 decoder. Its truth table is given below.

Truth table

3-bit binary input			Octal output							
A	B	C	O_0	O_1	O_2	O_3	O_4	O_5	O_6	O_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

From the truth table above, the expressions for octal outputs (for O_0 through O_7) can be written as following:

1. If the input is A=B=C=0 (LOW), Output O_0 will be HIGH. So, the output expression for O_0 is $O_0 = \overline{A}\overline{B}\overline{C}$. Similarly,

2. $O_1 = \overline{A}\overline{B}C$. 3. $O_2 = \overline{A}B\overline{C}$. 4. $O_3 = \overline{A}BC$ 5. $O_4 = A\overline{B}\overline{C}$

6. $O_5 = A\overline{B}C$ 7. $O_6 = AB\overline{C}$. 8. $O_7 = ABC$

The logic circuit is shown in Fig.11.

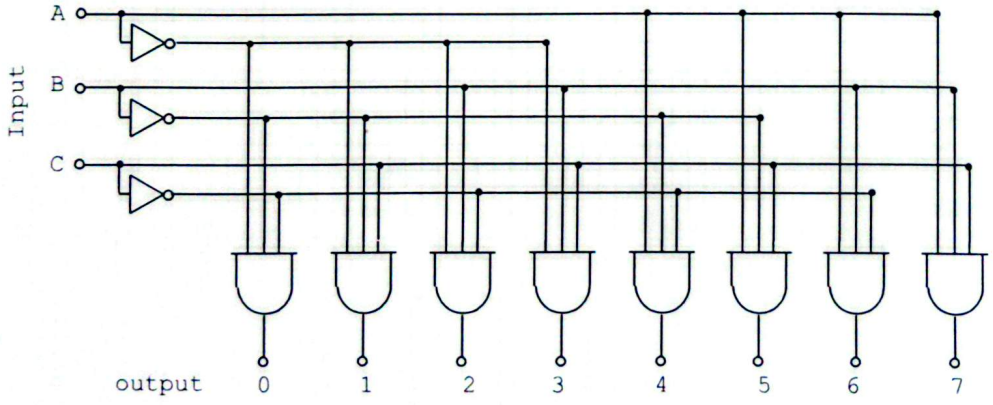


Fig.11: Binary-to-Octal decoder

7.2.3 BCD-to-7-Segment Decoder

A 7-segment LED display is a popular device commonly used for displaying decimal digits in many digital systems, such as in digital calculators, watches etc. This device consists of seven LED segments. These LED segments are generally defined by a, b, c, d, e, f, and g. The arrangement of 7 LED segments is shown below.

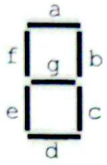


Fig.12: 7-LED-segments arrangement

For displaying decimal digit 0, segments a, b, c, d, e, and f will be ON. In similar way, for displaying 1, only segments b and c will be ON. Therefore, any decimal digit can be displayed using this display unit.

A BCD-to-7-segment decoder is a logic circuit that accepts decimal digits (0 through 9) in BCD format as inputs and converts it to a 7-segment code. For example, to display 5, the decoder accepts the BCD code 0101 as inputs and generates a 7-segment code so that only segments a, f, g, c, and d are ON which is nothing but . Fig.13 shows the block diagram of a BCD-to-7-segment display system.

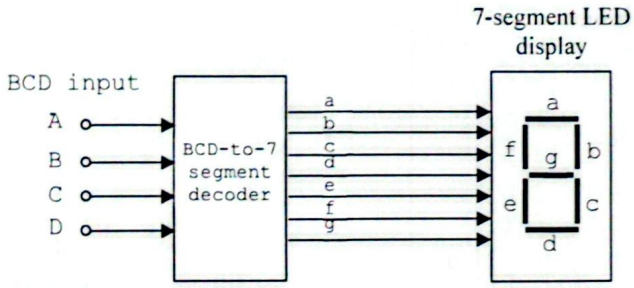


Fig.13: Block diagram of a BCD-to-7-segment display system

Truth table for BCD-to-7-segment decoder:

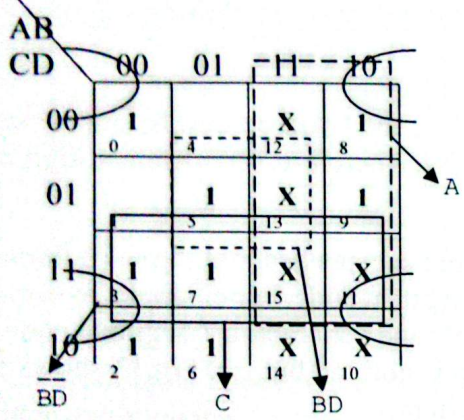
Decimal digit	7-segment decoder input				7-segment decoder output						
	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

CamScanner

Here for the decoder, ten BCD input combinations (0000 through 1001) are used. The remaining six BCD codes: 1010, 1011, 1100, 1101, 1110, and 1111 are unused and never appear in the inputs. Therefore, these unused codes may be treated as 'don't care' conditions ('x').

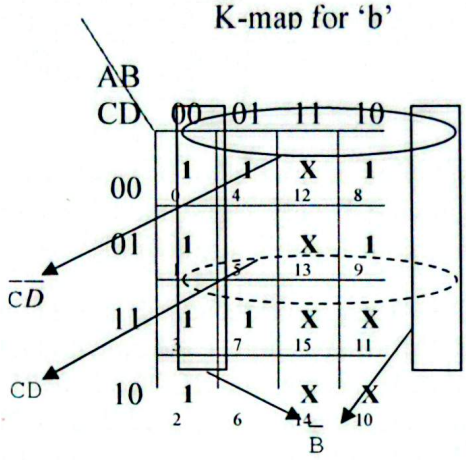
Simplification of the output expressions:

K-map for 'a'



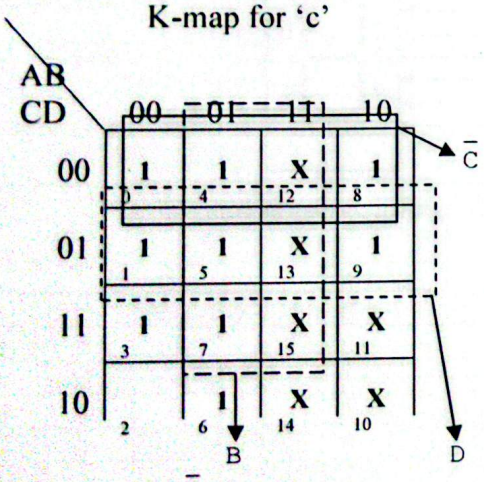
$\therefore$ $a=A+C+BD+\overline{B}\overline{D}$

K-map for 'b'



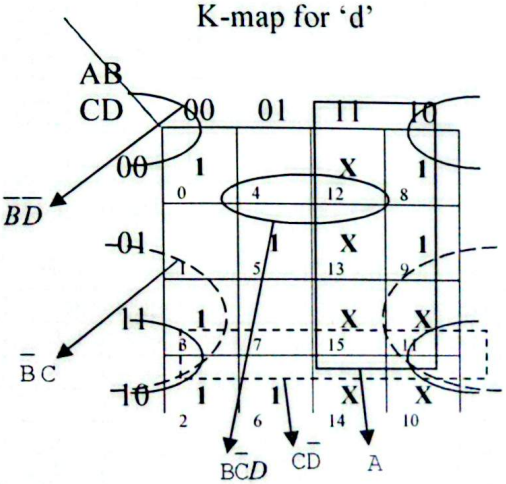
$\therefore b=\overline{B}+\overline{C}\overline{D}+CD$

K-map for 'c'



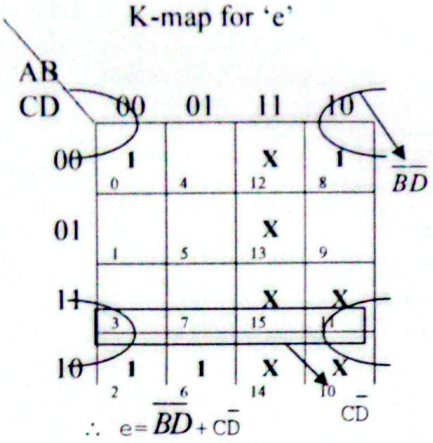
$\therefore$ $c=B+\overline{C}+D$

K-map for 'd'



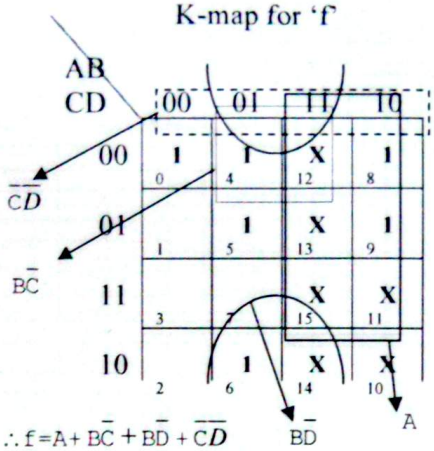
$\therefore d=A+\overline{B}\overline{D}+\overline{B}C+C\overline{D}+B\overline{C}D$

K-map for 'e'



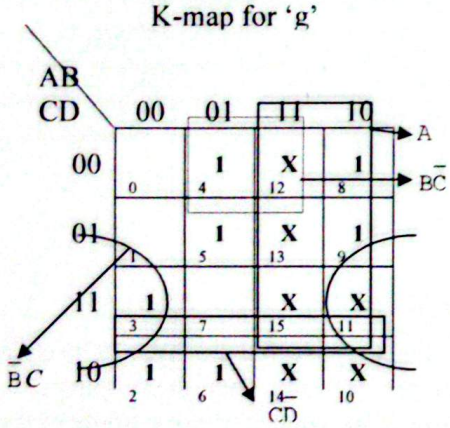
$\therefore e = \overline{BD} + \overline{CD}$

K-map for 'f'



$\therefore f = A + \overline{BC} + \overline{BD} + \overline{CD}$

K-map for 'g'



$\therefore g = A + \overline{BC} + \overline{B}C + \overline{CD}$

Therefore, the logic expressions for a, b, c, d, e, f, and g are:

$a = A + C + BD + \overline{BD}$

$b = \overline{B} + \overline{C}\overline{D} + CD$

$c = B + \overline{C} + D$

$d = A + \overline{BD} + \overline{B}C + C\overline{D} + B\overline{C}D$

$e = \overline{BD} + C\overline{D}$

$f = A + \overline{BC} + B\overline{D} + \overline{C}\overline{D}$

$g = A + \overline{BC} + \overline{B}C + C\overline{D}$

The logic circuit for BCD-to-7-segment decoder is shown in Fig.14.

CamScanner

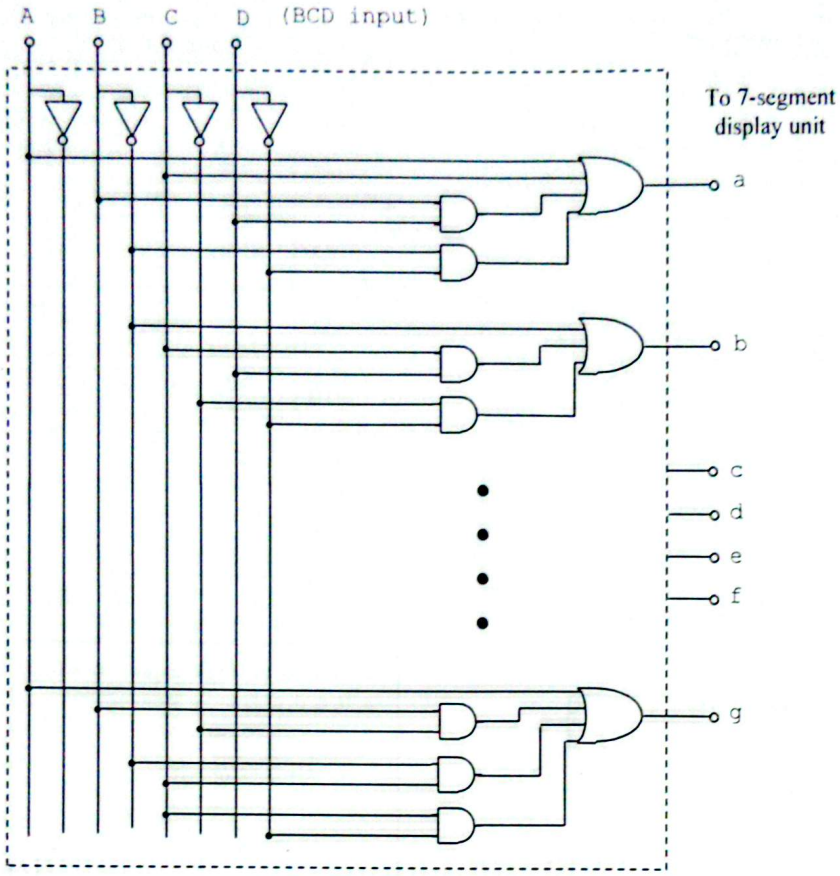


Fig.14: BCD-to-7-segment decoder

Operation: To illustrate the operation of the decoder, suppose that the input BCD code ABCD=1001 (which is 9 in decimal) is applied to the decoder. Then the outputs a, b, c, d, f, and g will be HIGH and e will be LOW. Now if we connect these outputs to the 7-segment display unit, it will display the digit as shown in Fig.15.

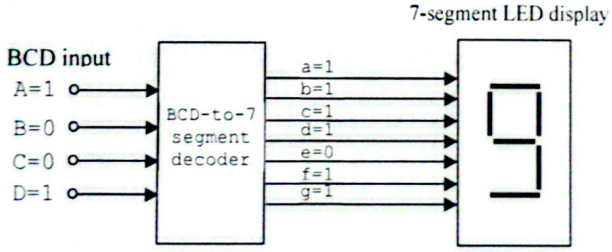


Fig.15

7.3 Liquid Crystal Displays

Another commonly used 7-segment display is the liquid crystal display. It also consists of seven LCD segments. Fig.16 shows the basic arrangement of the BCD-to-7-segment decoder and the LCD display.

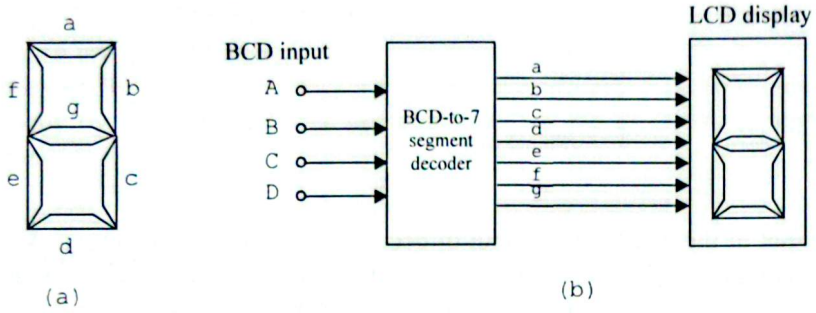


Fig.16: (a) Basic arrangement of 7-LCD segments
(b) Block diagram of the BCD-to-7-segment decoder and LCD display

Basic operation of an LCD segment

An LCD display controls the reflection of available light. When an LCD segment is activated, it does not reflect incident light and thus appears dark. On the other hand, a non-activated segment reflects incident light so that they appear invisible. Fig.17 and Fig.18 show the method for driving an LCD segment:

i) When an LCD segment is ON:

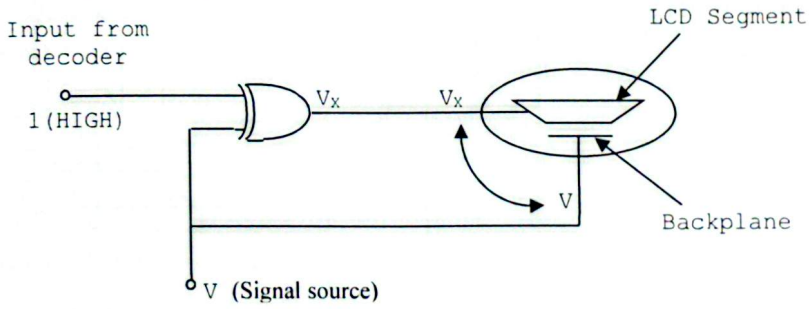


Fig.17

When there is a voltage difference between the backplane and the LCD segment, the segment will be ON and appears dark. Here it is seen that one input of the XOR gate (output of decoder) is HIGH. So, the output of the XOR gate, V_x is 180^0 out of phase with the signal source (V), i.e. for V=0, V_x will be 1 or vice-versa. Thus there always exists a voltage difference between the plane and the segment. Therefore, the segment is always ON and appears dark.

ii) When an LCD segment is OFF:

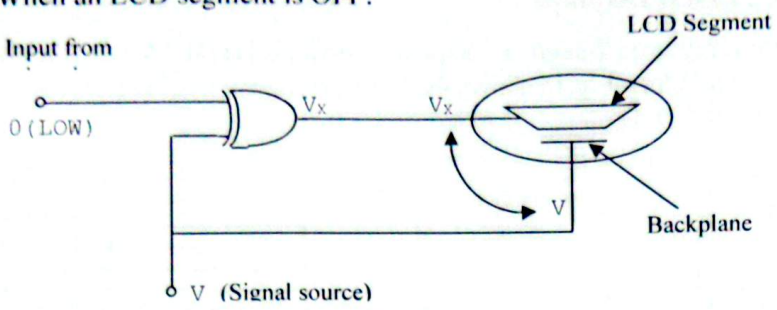


Fig.18

Here one input of the XOR gate (output of decoder) is LOW. So, its output, V_x is the same as signal source (V) and no voltage difference between the backplane and the segment exists. Hence, the segment remains OFF and appears invisible.

Therefore, it can be finalized that if the output of the decoder is 1, the segment will be ON and appear dark. Otherwise, the segment will remain OFF and appear invisible. A complete logic circuit for driving a 7-segment LCD display is shown in Fig.19.

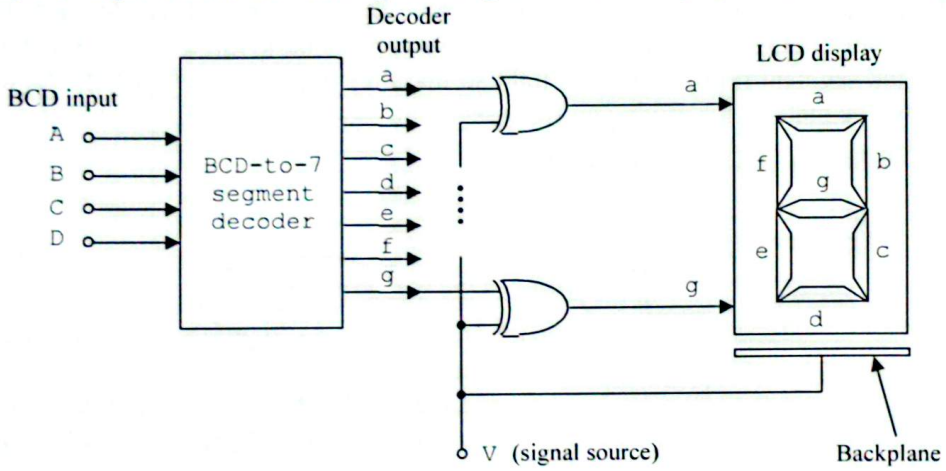
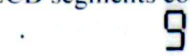


Fig.19: Block diagram of a BCD-to-7-segment decoder and LCD display

LCDs operate from a low-frequency signal voltage (30Hz to 60Hz) applied between the segment and the backplane. Each LCD segment is driven by an XOR gate with one input connected to an output of the decoder and the other input connected to the signal source. When the decoder output is HIGH (1), there develops a voltage difference between the segment and the backplane. Thus, the corresponding LCD segment is ON and appears dark.

To illustrate the operation of this logic circuit, suppose that the BCD input is 1001 (i.e. ABCD=1001 which is 9 in decimal) applied to the decoder. Then the decoder output will be a=1, b=1, c=1, d=1, e=0, f=1, and g=1. Thus the LCD segments connected to a, b, c, d, f, and g will be ON and it will display the digit . 

LCDs consume much less power compared to LEDs. But LEDs have a much brighter display which is easily visible in the dark. For driving 7-segment LCD display, CMOS devices are used, because they require much less power and the TTL low-state voltage is not exactly 0v, thus creating a dc component across the LCD which degrades its performance.

7.4 Larger Decoders

Larger decoder can be constructed using smaller ones. In that case, the enable inputs of smaller decoders are to be connected in a proper way and other inputs are common to all. For example, a 4-line-to-16-line decoder can be constructed using two 3-line-to-8-line decoders.

4-line-to-16-line Decoder using 3-line-to-8-line Decoders: The implementation of a 4-line-to-16-line decoder using 3-line-to-8-line decoders is shown in Fig.20. A 4-line-to-16-line decoder has 4 input lines (i.e. 4-bit input code) and 16 output lines. On the other hand, a 3-line-to-8-line decoder has 3 input lines and 8 output lines. Therefore, the enable input of 3-line-to-8-line decoder is to be used as the MSB of the 4-bit input code of the 4-line-to-16-line decoder and other inputs are common to both 3-line-to-8-line decoders.

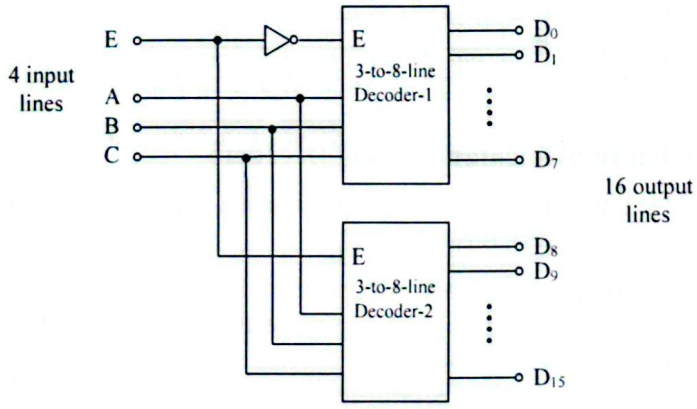


Fig.20: Implementation of 4-line-to-16-line decoder using two 3-line-to-8-line decoders

When E=0, decoder-1 is enabled and decoder-2 is disabled. So, all the outputs of decoder-2 are 0s. In this case, the input codes (EABC) are 0000, 0001, 0010, 0011, 0100, 0101, 0110, and 0111. For each of these input codes, only one of 8 outputs of decoder-1 will be activated.

When E=1, decoder-2 is enabled and decoder-1 is disabled. So, all the outputs of decoder-1 are 0s. In this case, the input codes (EABC) are 1000, 1001, 1010, 1011, 1100, 1101, 1110, and 1111. For each of these input codes, only one of 8 outputs of decoder-2 will be activated.

The operation of the 4-line-to-16-line decoder is summarized in the following truth table.

Truth table of a 4-line-to-16-line decoder

	Inputs				Outputs
	E	A	B	C	
Decoder-1 enable	0	0	0	0	D_0
	0	0	0	1	D_1
	0	0	1	0	D_2
	0	0	1	1	D_3
	0	1	0	0	D_4
	0	1	0	1	D_5
	0	1	1	0	D_6
	0	1	1	1	D_7
Decoder-2 enable	1	0	0	0	D_8
	1	0	0	1	D_9
	1	0	1	0	D_{10}
	1	0	1	1	D_{11}
	1	1	0	0	D_{12}
	1	1	0	1	D_{13}
	1	1	1	0	D_{14}
	1	1	1	1	D_{15}

7.5 Boolean Function Implementation using Decoders

Decoders can be used to implement any Boolean function of n-variables. The procedure is given below:

i. Express the Boolean function in standard SOP form,
ii. Use a decoder to generate the minterms of the function, and finally,
iii. Use an OR gate to form the sum.

Example 1. Implement the function $Y(A,B,C) = \overline{A}B + BC + \overline{A}C$ using a decoder.

Solution: i) Express the function in standard SOP form.

CamScanner

$$Y(A,B,C) = \overline{A}B + BC + \overline{A}C$$
$$= \overline{A}BC + \overline{A}B\overline{C} + ABC + \overline{A}\overline{B}C$$
$$= \sum m(1,2,3,7)$$

Since the function is of 3 variables, it requires a 3-line-to-8-line decoder to generate the minterms. Now OR the minterms. The implementation is shown in Fig.21.

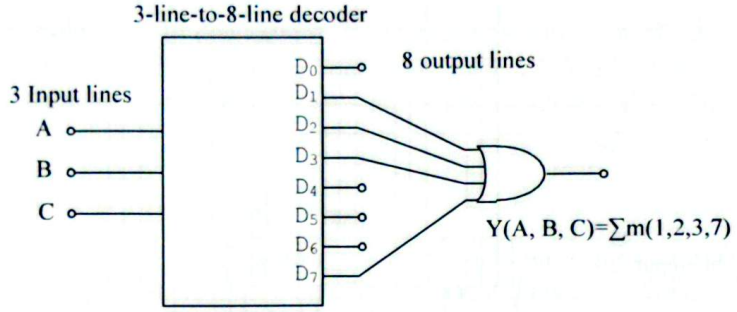


Fig.21: Implementation of logic function using DEMUX

Example 2. Implement a full adder logic circuit using decoders.
Solution: The output expressions for a full adder are

$$For\ Sum, S = \overline{A}B\overline{C_i} + A\overline{B}\overline{C_i} + \overline{A}\overline{B}C_i + ABC_i = \sum m(1,2,4,7)$$
$$For\ Carry, C_o = AB\overline{C_i} + \overline{A}BC_i + ABC_i + A\overline{B}C_i = \sum m(3,5,6,7)$$

The expressions are already in standard SOP form. Since there are 3 input variables (A, B, C_i), it requires a 3-line-to-8-line decoder to generate the minterms. Now

i) OR the minterms 1, 2, 4, and 7 for S and
ii) OR the minterms 3, 5, 6, and 7 for C_o.

The implementation is shown in Fig.22.

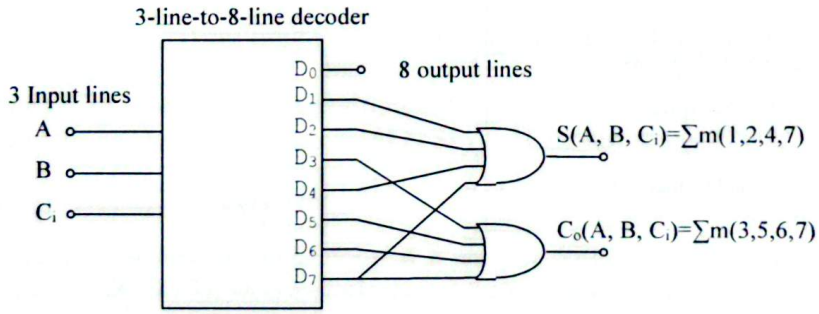


Fig.22: Implementation of full adder using decoder

7.6 Decoder as a DEMUX

Some decoders include one or more enable inputs to control the circuit operation. If the enable input is active, the output corresponding to the input code is activated. Otherwise, the outputs are disabled. Fig.23 shows a 3-line-to-8-line decoder with an enable input E.

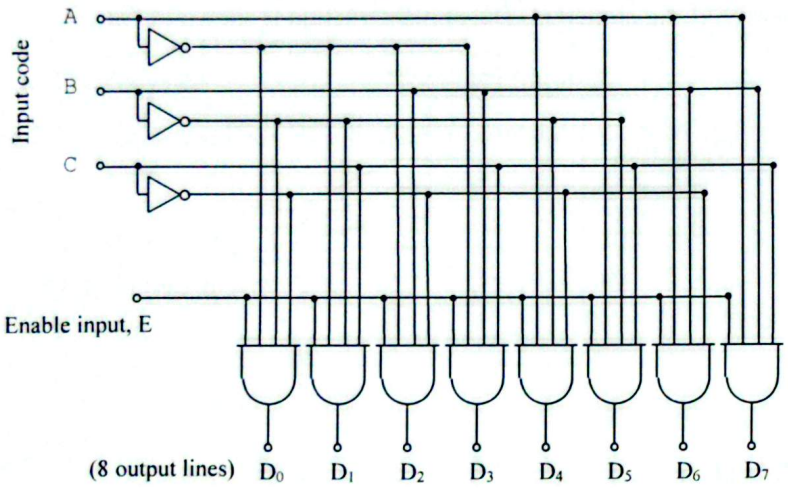


Fig.23: A 3-line-to-8-line decoder with an enable input, E

In this logic circuit if E=0 is applied, all the outputs are disabled. On the other hand, if E=1 is applied, the output corresponding to the input code (applied to ABC) is activated. That is, for E=1, the circuit will operate as a normal 3-line-to-8-line decoder.

A decoder with an enable input, E can be used as a DEMUX. In this case, the enable input, E is used as a data input line, I and the input code of the decoder is taken as the select input lines of the DEMUX. For example, the decoder of Fig.23 can be used as a 1:8 DEMUX.

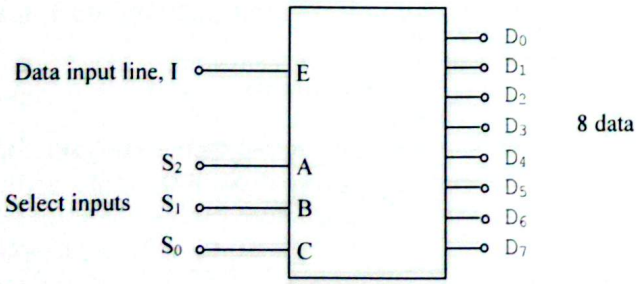


Fig.24: A 3-line-to-8-line decoder used as a 1:8 DEMUX.

For demultiplexing operation, E is to be used as data input line, I and the input code, ABC used as the select input lines, $S_2S_1S_0$. This is shown in Fig.24. Here the select input line will determine to which output line the data input I will be transmitted. For example, if the select input, $S_2S_1S_0$ is 110, the data input I will pass on to the data output line, D_6.

7.7 Applications of Decoders

There are many applications of decoders in digital systems. For example, decoders can be used to provide timing and sequencing operations; they are necessary in applications such as data multiplexing, 7-segment display, and memory address decoding.

7.8 Introduction to Multiplexers

In digital systems, it is often required to carry more than one signal on a single line. However, only one signal can be placed on a line at a time. Therefore, we need a device that will allow us to select different signals at different instant of time. This device is known as a multiplexer (or simply MUX). It is a widely used logic circuit in digital systems. It has several DATA input lines, some SELECT input lines and a single output line. Fig.25 shows a general block diagram of a multiplexer.

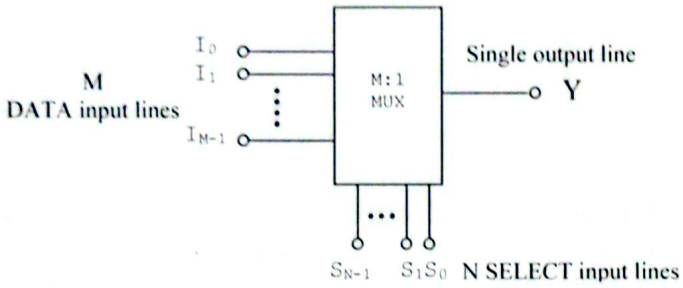


Fig.25: A general block diagram of a multiplexer

The SELECT input lines (N) are used to select only one of M DATA input lines at a time to pass the data from that DATA input line to the output, Y. Hence, multiplexers are also known as DATA selectors. A multiplexer is often abbreviated as MUX. The relation between the M DATA input lines and N SELECT input lines is $2^N \geq M$.

7.8.1 Two-Input (2:1) Multiplexer

A two-input multiplexer has two DATA input lines, one SELECT input line and a common output line. Fig.26 shows the block diagram of a 2-input multiplexer.

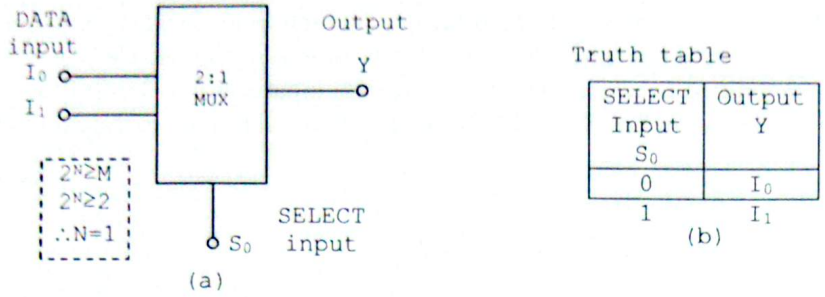


Fig.26: (a) Block diagram of a two-input multiplexer
(b) Its truth table

From the above truth table, the expression for output Y can be written as

$$Y = \overline{S}_0 I_0 + S_0 I_1$$

The logic circuit for the two-input multiplexer is shown in Fig.27.

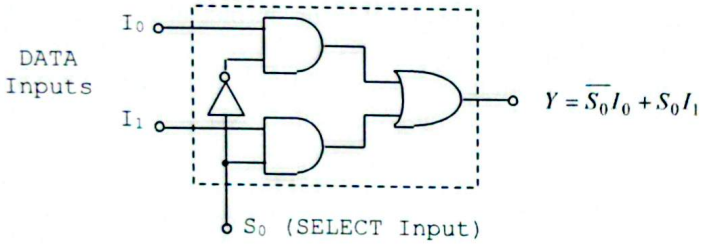


Fig.27: Two-input multiplexer

Operation: The SELECT input S_0 is used to select one of the two DATA input lines at a time, so that, the data from the selected DATA input line may pass on to the output Y. Here it is assumed that when the SELECT input S_0 is 0, DATA input line I_0 will be selected. So, data from I_0 will pass on to the output. When S_0 is 1, DATA input line I_1 will be selected and data from I_1 will pass on to the output.

7.8.2 Four-Input (4:1) Multiplexer

A four-input multiplexer has four DATA input lines, two SELECT input lines and a common output line. Fig.28 shows the block diagram and truth table of a four-input multiplexer.

CamScanner

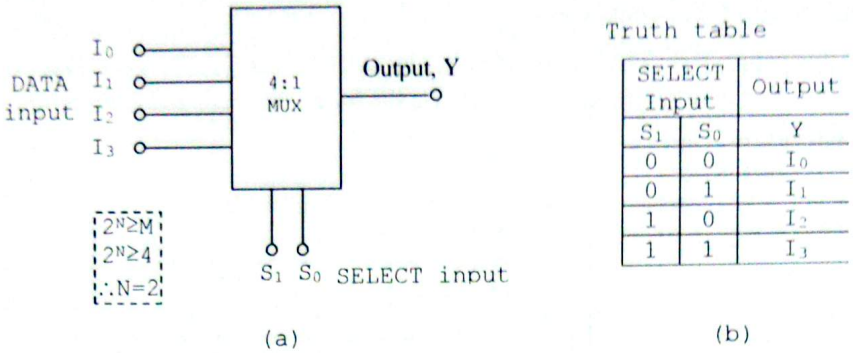


Fig.28: (a) Block diagram of a four-input multiplexer
(b) Its truth table

From the above truth table, the expression for output Y can be written as

$$Y = \overline{S}_1 \overline{S}_0 I_0 + \overline{S}_1 S_0 I_1 + S_1 \overline{S}_0 I_2 + S_1 S_0 I_3$$

The logic circuit of the multiplexer is shown in Fig.29.

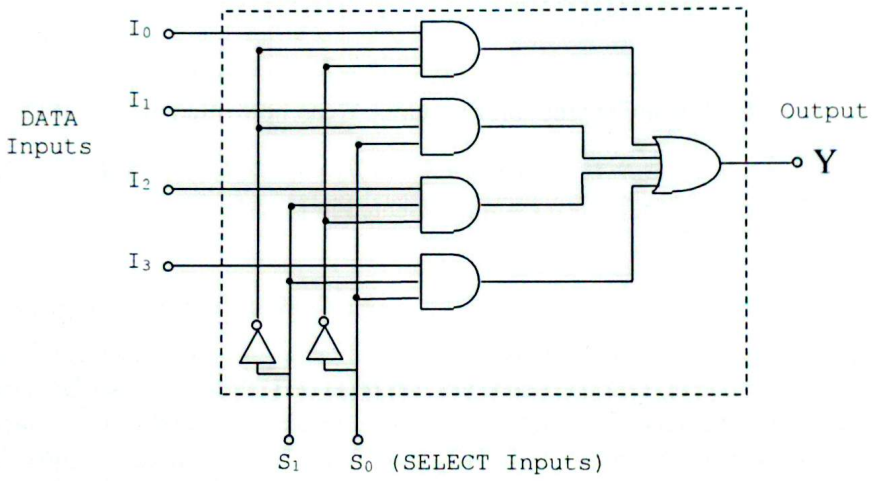


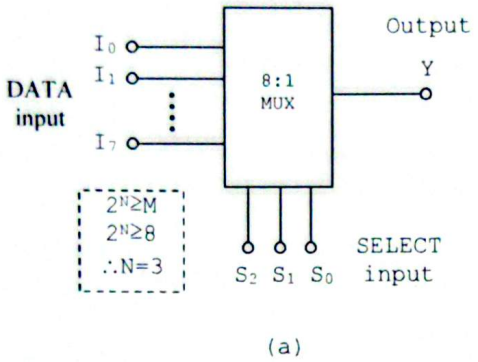
Fig.29: Logic circuit of a four-input multiplexer

Operation: The two SELECT inputs S_1, S_0 are used to select one of the four DATA input lines at a time, so that, the data from the selected DATA input line may pass on to the output Y. When S_1=S_0=0, DATA input line I_0 will be selected. So, data from I_0 will pass on to the output. Similarly, for S_1=0 and S_0=1, I_1 will be selected; for S_1=1 and S_0=0, I_2 will be selected; and for S_1=1 and S_0=1, I_3 will be selected.

CamScanner

7.8.3 Eight-Input (8:1) Multiplexer

This multiplexer has eight DATA input lines. To select eight DATA input lines, three SELECT input lines are required. Fig.30 shows the block diagram and truth table of an eight-input multiplexer.



(a)

Truth table

SELECT Input			Output
S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

(b)

Fig.30: (a) Block diagram of an eight-input multiplexer
(b) Its truth table

From the above truth table, the expression for output Y can be written as

$$Y = \bar{S}_2\bar{S}_1\bar{S}_0I_0 + \bar{S}_2\bar{S}_1S_0I_1 + \bar{S}_2S_1\bar{S}_0I_2 + \bar{S}_2S_1S_0I_3 + S_2\bar{S}_1\bar{S}_0I_4 + S_2\bar{S}_1S_0I_5 + S_2S_1\bar{S}_0I_6 + S_2S_1S_0I_7$$

The logic circuit of the 8:1 multiplexer is implemented in Fig.31.

CamScanner

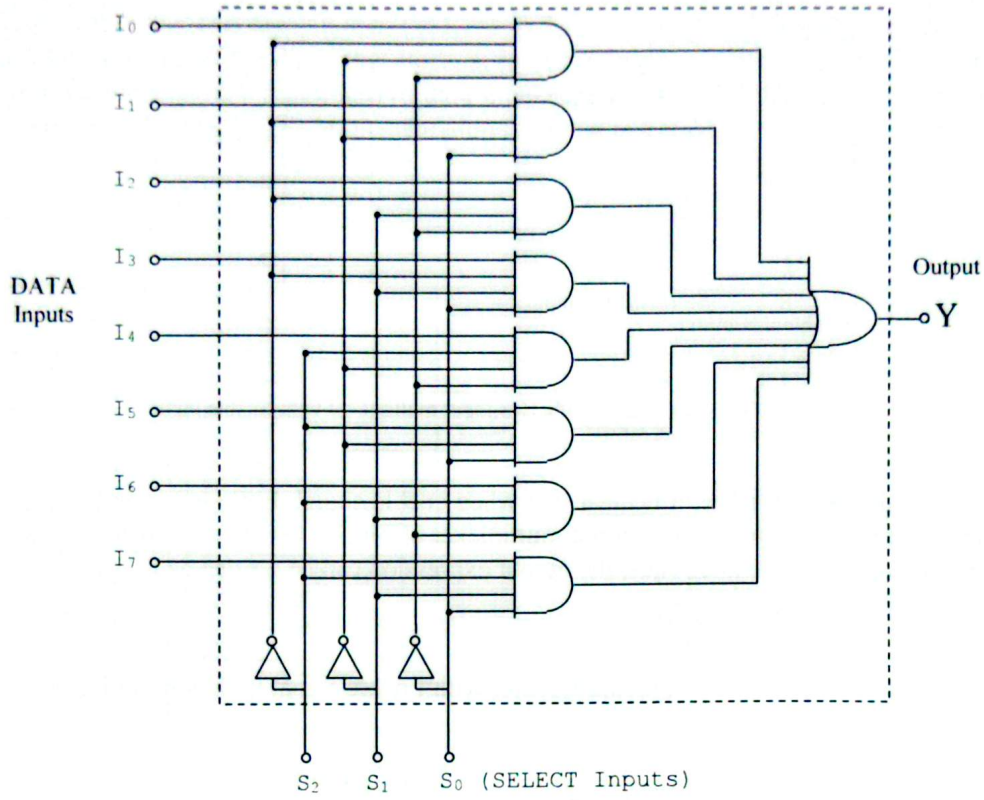


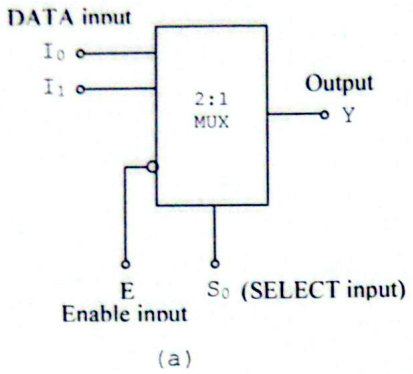
Fig.31: Logic circuit of an eight-input multiplexer

7.9 Multiplexer with Enable Signal

Multiplexer ICs may have an additional input signal known as Enable signal (E) to control the operation of the circuit. When the enable signal is in active state, the circuit performs as a normal multiplexer. However, when it is in the other state, the outputs are disabled. The enable input (normally active-LOW) can also be used to expand two or more multiplexer ICs to a multiplexer with a larger number of inputs.

To describe how the enable signal (E) controls the operation of a multiplexer, let us consider the block diagram and truth table of a two-input multiplexer with active-LOW enable signal.

CamScanner



Truth table

E	S_0	Output, Y
0	0	I_0
0	1	I_1
1	0	0
1	1	0

(b)

Fig.32: (a) Block diagram of a two-input multiplexer with Enable input
(b) Its truth table

From the above truth table, it is clear that when enable signal, E=0 (active state), the circuit performs as a normal two-input multiplexer of Fig.26. On the other hand, when E=1, the output is always 0. Thus, the output expression can be written as follows:

$$Y = \overline{E}\overline{S}_0 I_0 + \overline{E}S_0 I_1$$

The logic circuit of the two-input multiplexer with Enable signal is shown in Fig.33.

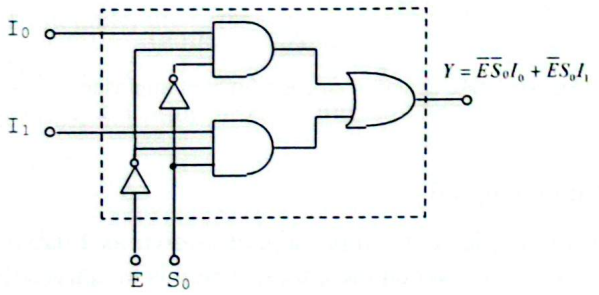


Fig.33: A two-input multiplexer with Enable signal

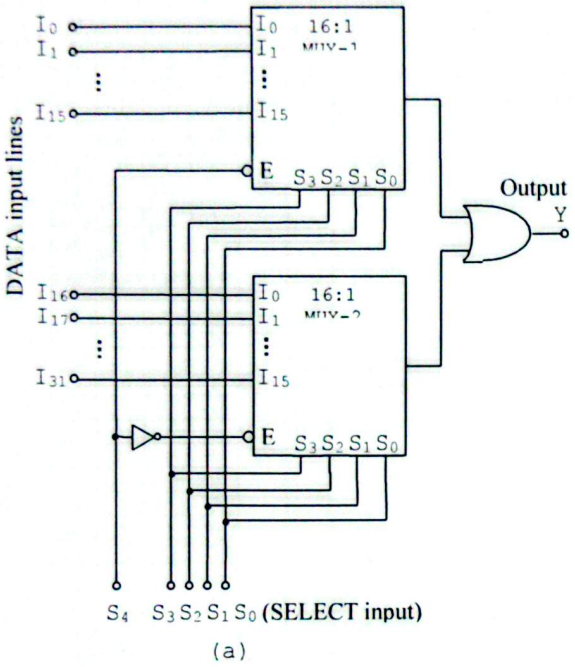
7.10 Larger MUXs

Larger multiplexers can be constructed using smaller ones. For example, a 32:1 MUX can be constructed with two 16:1 MUXs, an 16:1 MUX with two 8:1 MUXs, and so on.

7.10.1 32:1 MUX using 16:1 MUXs

CamScanner

A 32:1 MUX has 32 data input lines ($I_0, I_1, \ldots\ldots, I_{31}$) and one output line (Y). Therefore, to select any one of the data input lines, at least 5 select input lines ($S_0, \ldots, S_4$) are required. Since a 16:1 MUX has 16 data input lines, two 16:1 MUXs are needed to construct a 32:1 MUX. The implementation is shown in Fig.34.



Truth table

	S_4	S_3	S_2	S_1	S_0	Y
MUX-1 Enable	0	0	0	0	0	I_0
	0	0	0	0	1	I_1
	⋮	⋮	⋮	⋮	⋮	⋮
	0	1	1	1	1	I_{15}
MUX-2 Enable	1	0	0	0	0	I_{16}
	1	0	0	0	1	I_{17}
	⋮	⋮	⋮	⋮	⋮	⋮
	1	1	1	1	1	I_{31}

(b)

Fig.34: (a) Implementation of a 32:1 MUX using 16:1 MUXs
(b) Its truth table

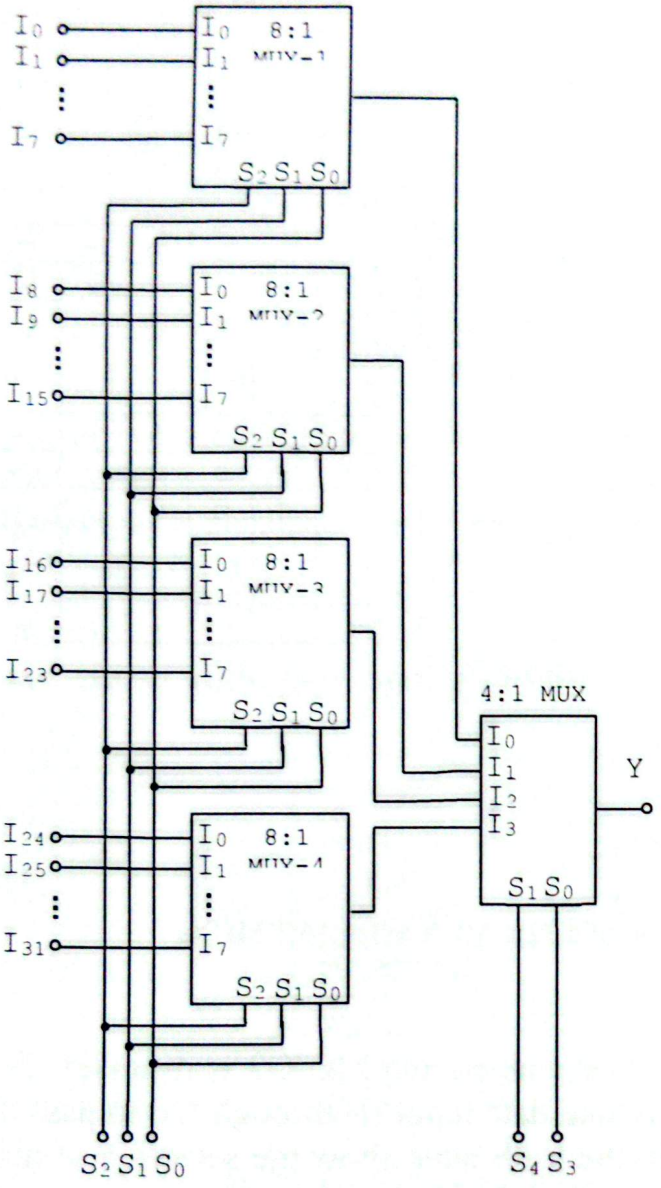
I) When S_4=0: In this case, MUX-1 is enabled and MUX-2 is disabled. The select input lines ($S_0, \ldots, S_4$) will allow only one data input (I_0 through I_{15}) to pass on to the output at a time. The first 16 rows in the truth table show the selection of data input lines of MUX-1.

II) When S_4=1: In this case, MUX-2 is enabled and MUX-1 is disabled. Therefore, the select input lines ($S_0, \ldots, S_4$) will select only one data input (I_{16} through I_{31}) of MUX-2 at a time to pass on to the output. The last 16 rows in the truth table show the selection of data input lines of MUX-2.

7.10.2 32:1 MUX using 8:1 and 4:1 MUXs

CamScanner

A 32:1 MUX has 32 data input lines (I_0, I_1, ……, I_{31}), 5 select input lines (S_0, …, S_4), and one data output line (Y). Since an 8:1 MUX has 8 data input lines, four 8:1 MUXs are required to construct a 32:1 MUX. Again, to select one of four 8:1 MUXs at a time, another 4:1 MUX is required. The implementation is shown in Fig.35.



Truth table

	S_4	S_3	S_2	S_1	S_0	Y
MUX-1 Enable	0	0	0	0	0	I_0
			0	0	1	I_1
			0	1	0	I_2
			0	1	1	I_3
			1	0	0	I_4
			1	0	1	I_5
			1	1	0	I_6
			1	1	1	I_7
MUX-2 Enable	0	1	0	0	0	I_8
			0	0	1	I_9
			0	1	0	I_{10}
			0	1	1	I_{11}
			1	0	0	I_{12}
			1	0	1	I_{13}
			1	1	0	I_{14}
			1	1	1	I_{15}
MUX-3 Enable	1	0	0	0	0	I_{16}
			0	0	1	I_{17}
			0	1	0	I_{18}
			0	1	1	I_{19}
			1	0	0	I_{20}
			1	0	1	I_{21}
			1	1	0	I_{22}
			1	1	1	I_{23}
MUX-4 Enable	1	1	0	0	0	I_{24}
			0	0	1	I_{25}
			0	1	0	I_{26}
			0	1	1	I_{27}
			1	0	0	I_{28}
			1	0	1	I_{29}
			1	1	0	I_{30}
			1	1	1	I_{31}

(b)

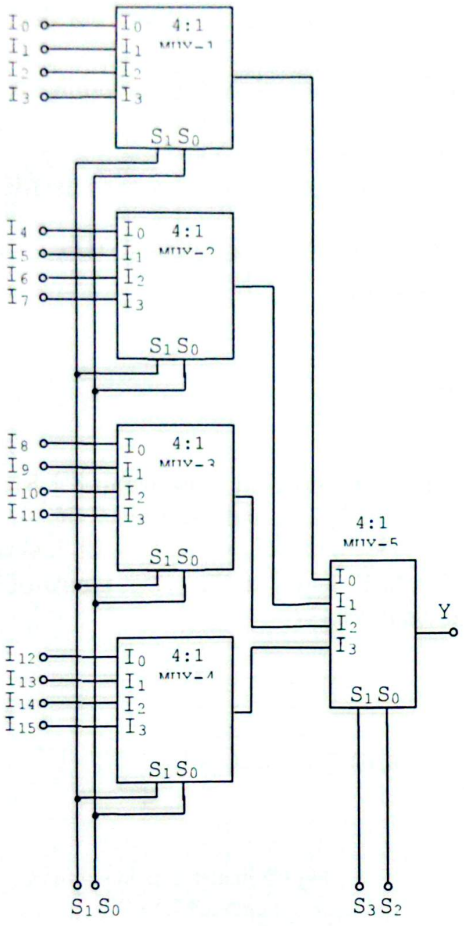
Fig.35: (a) Implementation of a 32:1 MUX using 8:1 and 4:1 MUXs
(b) Its truth table

The operation of this logic circuit can be explained with an example. Suppose that the select input $S_4S_3S_2S_1S_0$=01000 is applied. Here $S_2S_1S_0$=000 will select the first data input line I_0 of each 8:1 MUX. However, S_4S_3=01 will allow only I_0 of MUX-2 (which

is nothing but I_8) to pass on to the output, Y. Thus, I_8 of 32 data input lines will appear at the output finally as shown in the truth table.

7.10.3 16:1 MUX using 4:1 MUXs

A 16:1 MUX has 16 data input lines (I_0, I_1, ……, I_{15}), 4 select input lines (S_0, …, S_3), and one data output line (Y). Since a 4:1 MUX has 4 data input lines, four 4:1 MUXs are required. Again, to select one of four 4:1 MUXs at a time, another 4:1 MUX is needed. Thus, it requires a total of five 4:1 MUXs to construct a 16:1 MUX. The implementation is shown in Fig.36.



(a)

Truth table

	S_3	S_2	S_1	S_0	Y
MUX-1 Enable	0	0	0	0	I_0
			0	1	I_1
			1	0	I_2
			1	1	I_3
MUX-2 Enable	0	1	0	0	I_4
			0	1	I_5
			1	0	I_6
			1	1	I_7
MUX-3 Enable	1	0	0	0	I_8
			0	1	I_9
			1	0	I_{10}
			1	1	I_{11}
MUX-4 Enable	1	1	0	0	I_{12}
			0	1	I_{13}
			1	0	I_{14}
			1	1	I_{15}

(b)

Fig.36: (a) Implementation of a 16:1 MUX using 4:1 MUXs
(b) Its truth table

7.11 Boolean Function Implementation using MUXs

Any Boolean function of n-variables can be implemented using either 2^n-to-1 line or 2^{n-1}-to-1 line multiplexer. In fact, using multiplexers in the design of these logic circuits have some advantages. For example, it does not require simplification of logic expressions; it minimizes the IC package count; and it simplifies the logic design.

I. Using 2^n-to-1 line multiplexers

Any Boolean function of n-variables can be realized using 2^n-to-1 line multiplexers. The implementation procedure is as follows:

i. Express the function in standard SOP form and determine the decimal number corresponding to each minterm.
ii. Apply logic 1 to the data input lines of the MUX corresponding to the decimal numbers obtained above.
iii. Apply logic 0 to all other data input lines of the MUX.
iv. Connect the input variables of the function to the select inputs of the MUX.

Example 3. Implement the function $Y(A,B,C) = \overline{A}B + BC + \overline{A}C$ using a MUX.

Solution: i) Express the function in standard SOP form.

$$Y(A,B,C) = \overline{A}B + BC + \overline{A}C$$
$$= \overline{A}BC + \overline{A}B\overline{C} + ABC + \overline{A}\overline{B}C$$
$$= \sum m(1,2,3,7)$$

ii) Since there are three input variable A, B, and C, it requires an 8:1 MUX. Now apply logic 1 to the data input lines 1, 2, 3, and 7 of the MUX.
iii) All other data input lines are logic 0.
iv) Finally, connect the input variables A, B, and C to the select inputs of the MUX. The implementation is shown in Fig.37.

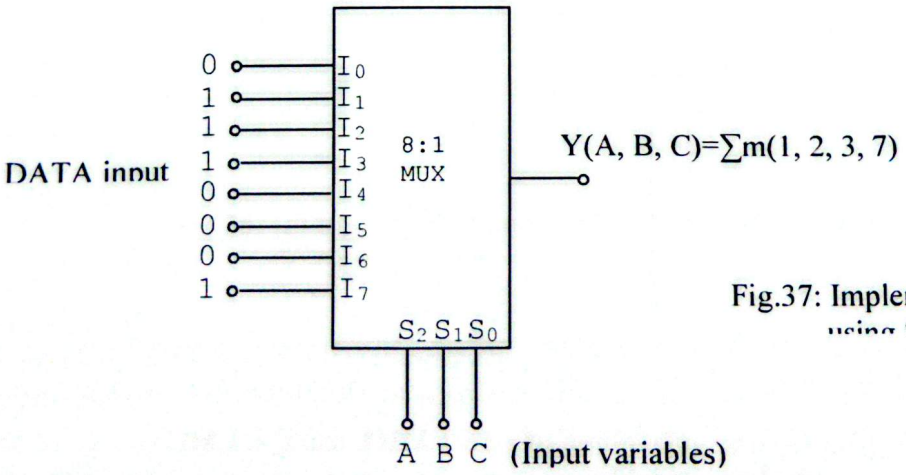


Fig.37: Implementation of function using 8:1 MUX

Example 4. Implement the following function using a MUX

$Y(A, B, C, D)=\sum m(0, 1, 2, 3, 5, 7, 8, 10, 12, 13, 15)$

Solution: i) The function is already in standard SOP form.

ii) Since there are four input variable A, B, C, and D, it requires a 16:1 MUX. Now apply logic 1 to the data input lines 0, 1, 2, 3, 5, 7, 8, 10, 12, 13, and 15 of the MUX.

iii) All other data input lines are logic 0 and finally,

iv) Connect the input variables A, B, C, and D to the select inputs of the MUX.

The implementation is shown in Fig.38.

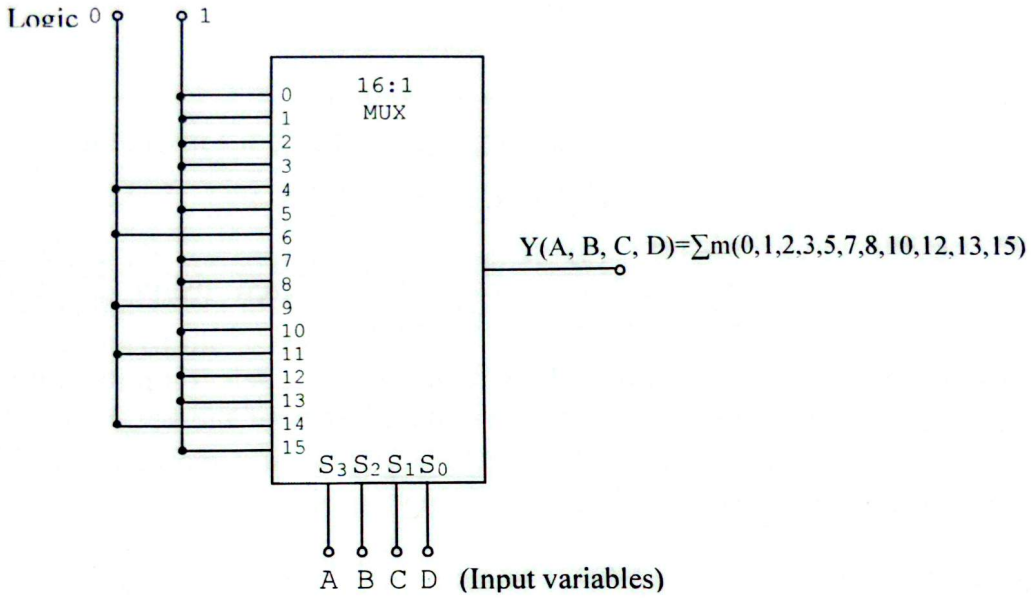


Fig.38: Implementation of function using 16:1 MUX.

Example 5. Implement the function specified in table-1 using MUX.

Table-1

Inputs			Output,
A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Solution: The function can be written in standard SOP form as

$Y(A, B, C)=\sum m(1, 3, 5, 6)$

The implementation is shown in Fig.39.

CamScanner

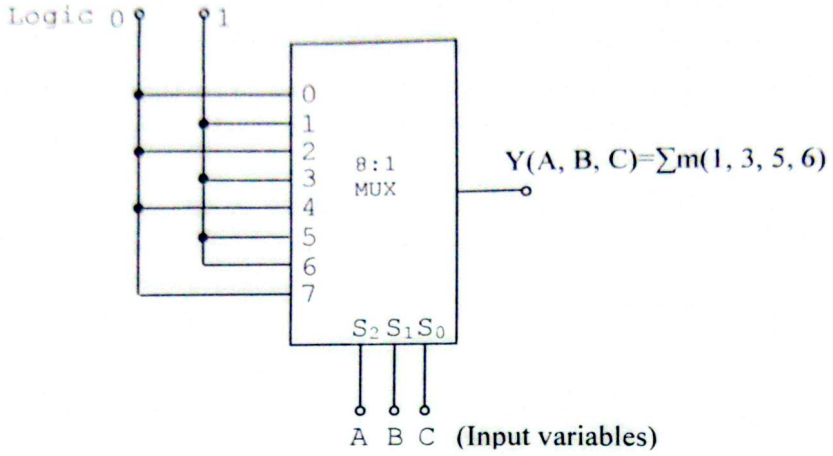


Fig.39: Implementation of function using 8:1 MUX.

II. Using 2^{n-1}-to-1 line multiplexers

Any Boolean function of n-variables can also be realized using 2^{n-1}-to-1 line multiplexers. The implementation procedure is as follows:

i. Choose the first n-1 variables of the function as the select inputs of the MUX and use the remaining variable for the data inputs of the MUX.
ii. Now construct the truth table for the function and express the function output in terms of 0, 1, and the remaining variable.

Example 6. Implement the following function using 4:1 MUX

$Y(A, B, C)=\sum m(0, 1, 3, 6)$

Solution: This is a function of 3 input variables. We may choose the first two variables A and B as the select inputs of the 4:1 MUX. The remaining variable (C or C′) is used for the data inputs of the MUX.

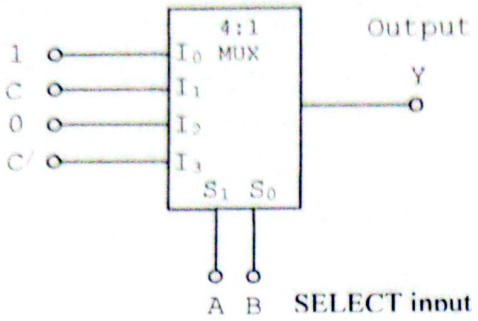
Now we construct the truth table for the function and express the output values in terms of 0, 1, C, and C′ as follows.

Truth table

A	B	C	Y	Express Y in terms of 0, 1, C and C′
0	0	0	1	Y=1
		1	1	
0	1	0	0	Y=C
		1	1	
1	0	0	0	Y=0
		1	0	
1	1	0	1	Y=C′
		1	0	

Implementation of the function:

CamScanner



Truth table redrawn

A	B	Y
0	0	1
0	1	C
1	0	0
1	1	C′

Fig.40: Implementation of function using 4:1 MUX

Example 7. Implement the following function using 8:1 MUX

$$Y(A, B, C, D)=\sum m(0, 2, 4, 5, 6, 8, 10, 13)$$

Solution: This is a function of 4 input variables. We may choose the first three variables A, B, and C as the select inputs of the MUX. The remaining variable (D or D′) is used for the data inputs of the MUX.

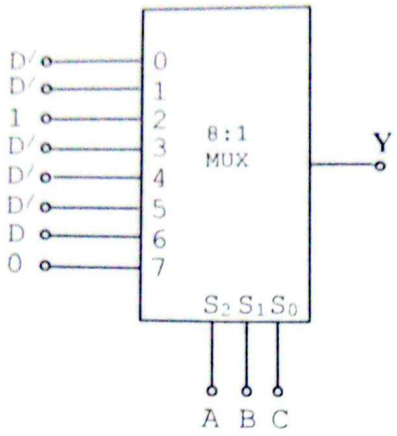
Now we construct the truth table for the function and express the output values in terms of 0, 1, D, and D′ as follows.

Truth table

A	B	C	D	Y	Express Y in terms of 0, 1, D and D′
0	0	0	0	1	Y=D′
			1	0	
0	0	1	0	1	Y=D′
			1	0	
0	1	0	0	1	Y=1
			1	1	
0	1	1	0	1	Y=D′
			1	0	
1	0	0	0	1	Y=D′
			1	0	
1	0	1	0	1	Y=D′
			1	0	
1	1	0	0	0	Y=D
			1	1	
1	1	1	0	0	Y=0
			1	0	

Implementation of the function using 8:1 MUX is shown in Fig.41.

CamScanner



Truth table redrawn

A	B	C	Y
0	0	0	D/
0	0	1	D/
0	1	0	1
0	1	1	D/
1	0	0	D/
1	0	1	D/
1	1	0	D
1	1	1	0

Fig.41: Implementation of the function using 8:1 MUX.

Example 8. Implement the following Boolean function using a MUX

$F(A, B, C, D)=\sum m(0, 1, 3, 4, 8, 9, 15)$

Solution: This is a function of 4 input variables. So, this function can be realized using either a 16:1 MUX or an 8:1 MUX.

I. Using a 16:1 MUX

Since this function is already in standard SOP form,

i) Apply logic 1 to data input lines 0, 1, 3, 4, 8, 9, and 15 of the MUX.
ii) Apply logic 0 to other data input lines, and
iii) Connect the input variables of the function *F* to the select input lines of the 16:1 MUX.

Following the above three steps, the implementation of the function F is shown in Fig.42.

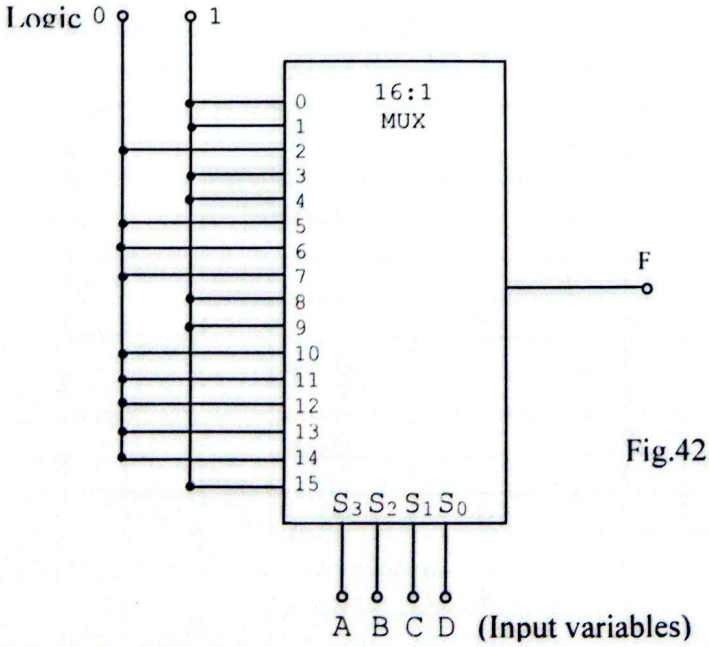


Fig.42: Implementation of the function, F using a 16:1 MUX.

II. Using an 8:1 MUX

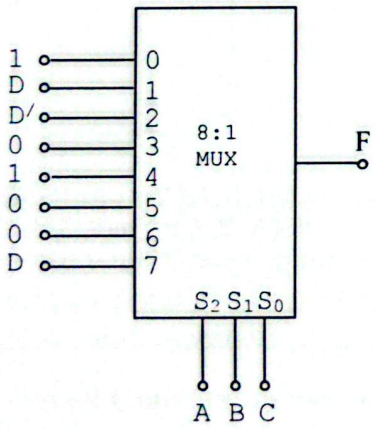
The above function can also be realized using an 8:1 MUX. Since the function is of 4 input variables. We may choose the first three variables A, B, and C as the select inputs of the MUX. The remaining variable (D or D′) is used for the data inputs of the MUX.

Now we construct the truth table for the function and express the output values in terms of 0, 1, D, and D′ as follows:

Truth table

A	B	C	D	F	Express F in terms of 0, 1, D and D′
0	0	0	0	1	F=1
			1	1	
0	0	1	0	0	F=D
			1	1	
0	1	0	0	1	F=D′
			1	0	
0	1	1	0	0	F=0
			1	0	
1	0	0	0	1	F=1
			1	1	
1	0	1	0	0	F=0
			1	0	
1	1	0	0	0	F=0
			1	0	
1	1	1	0	0	F=D
			1	1	

Implementation of the function, F using 8:1 MUX is shown in Fig.43.



Truth table redrawn

A	B	C	F
0	0	0	1
0	0	1	D
0	1	0	D′
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	D

Fig.43: Implementation of the function, F using 8:1 MUX.

Example 9. Implement the following functions with multiplexers

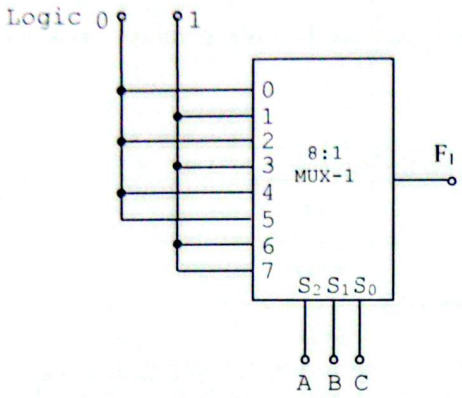
$F_1(A, B, C) = \sum m(1, 3, 6, 7)$

CamScanner

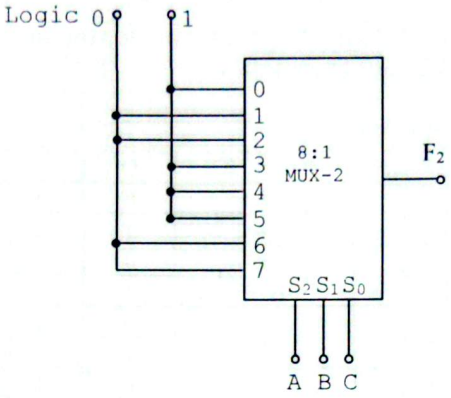
$F_2(A, B, C) = \sum m(0, 3, 4, 5)$

Solution: Each function is of 3 input variables. So, each requires one 8:1 MUX as shown below:

Implementation of function, F_1



Implementation of function, F_2



Combining both implementations of F_1 and F_2 we have,

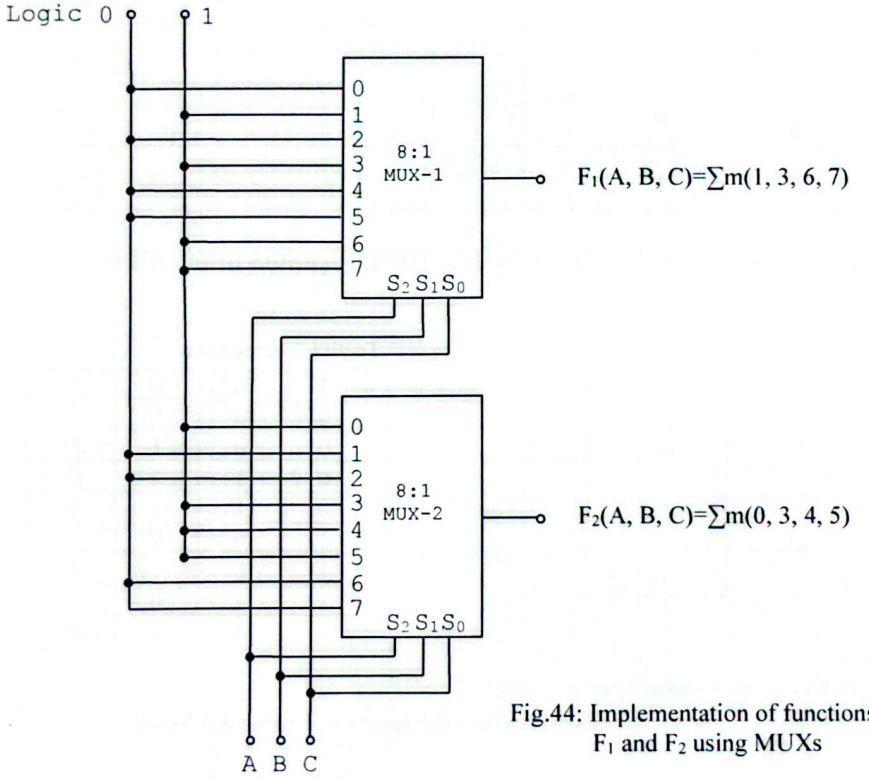


Fig.44: Implementation of functions F_1 and F_2 using MUXs

Example 10: Implement a full adder logic circuit using multiplexers.

Solution: The output expressions for a full adder are

$$For\ Sum, S = \overline{A}B\overline{C_i} + A\overline{B}\overline{C_i} + \overline{A}\overline{B}C_i + ABC_i = \sum m(1,2,4,7)$$

$$For\ Carry, C_o = AB\overline{C_i} + \overline{A}BC_i + ABC_i + A\overline{B}C_i = \sum m(3,5,6,7)$$

These expressions are of 3 input variables. So, it requires two 8:1 MUXs for S and C_o.

The implementation is shown in Fig.45.

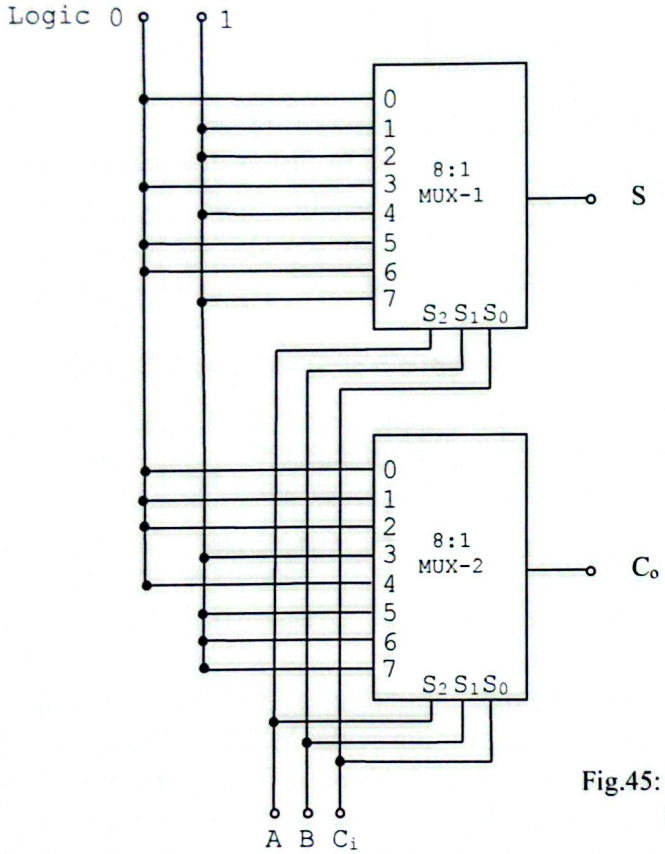


Fig.45: Implementation of a full adder using MUXs

7.12 Applications of Multiplexers

Multiplexers are widely used in many digital systems. Some of the major applications of multiplexers are parallel-to-serial data conversion, data routing, data selection, wave form generation, and operation sequencing.

Parallel-to-Serial Conversion using MUX: In digital systems, binary data are usually processed in parallel form because, it is faster. However, when data are to be transmitted over relatively long distances, the parallel arrangement is undesirable because, it requires a large number of transmission lines. For this reason, parallel data are converted to serial form. The conversion can be done with a multiplexer. Fig.46 shows this conversion process using a MUX.

CamScanner

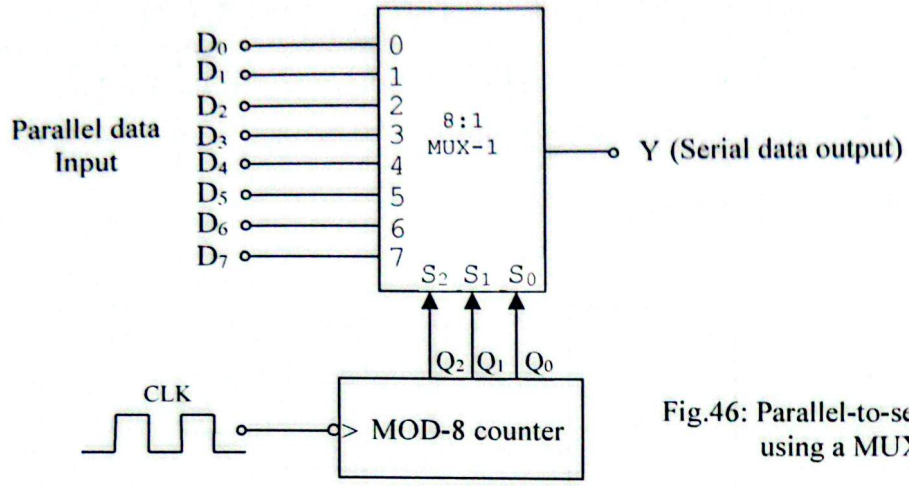
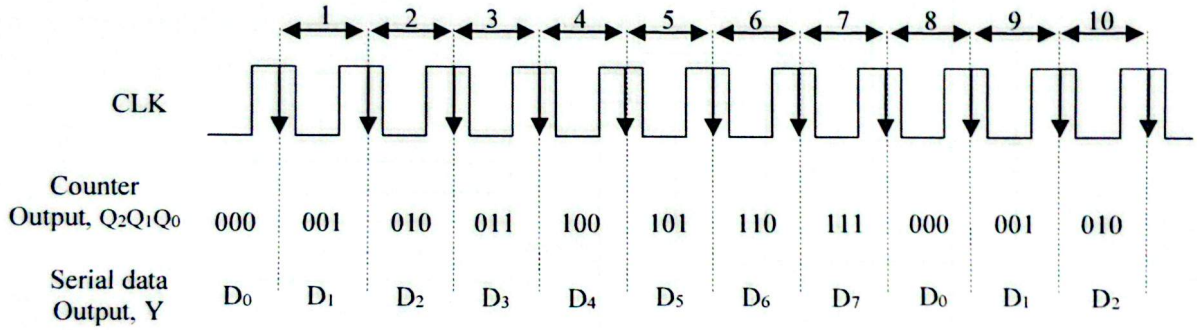


Fig.46: Parallel-to-serial data conversion using a MUX

Pulsed operation:



The data are present in parallel form at the data input lines of the MUX. The outputs of a MOD-8 counter are connected to the select inputs of the MUX. As the clock pulses are applied to the counter, the values of $S_2S_1S_0$ cycle through 000 to 111. When $S_2S_1S_0$ is 000, the data D_0 passes onto the output, Y; then when $S_2S_1S_0$ is 001, the data D_1 passes onto the output, Y, and so on. Therefore, the output, Y is the serial representation of the parallel input data ($D_0D_1....D_7$).

7.13 Introduction to Demultiplexer

A demultiplexer (DEMUX) is a logic circuit that performs the reverse operation of a multiplexer. It receives data from a single input line and distributes it over several outputs. Fig.47 shows a general block diagram of a demultiplexer.

CamScanner

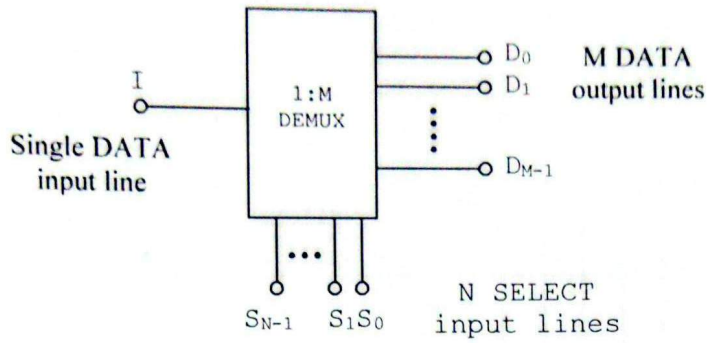
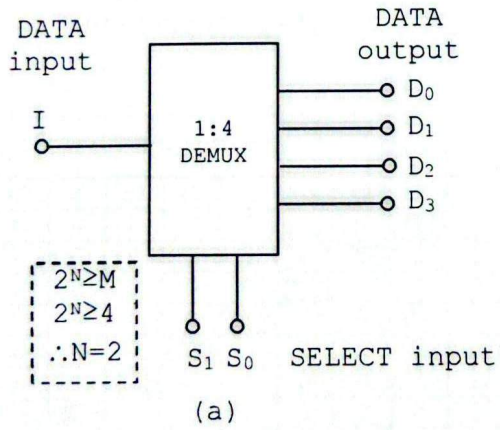


Fig.47: A general block diagram of a demultiplexer

The SELECT input lines N determine to which output line, the DATA input I will be transmitted. For this reason, a demultiplexer is also known as a data distributor. The relation between the output lines M and SELECT input lines N is $2^N \geq M$.

7.13.1 1-line-to-4-line (1:4) Demultiplexer

A 1-line-to-4-line demultiplexer has a single DATA input line and 4 DATA output lines (i.e. M=4). To select four DATA output lines, two SELECT input lines are required. Fig.48 (a) shows the block diagram of a 1-line-to-4-line demultiplexer. Suppose when S_1S_0=00, the DATA input I will appear at output line D_0; when S_1S_0=01, I will appear at D_1. Similarly, when S_1S_0=10, I will appear at D_2 and when S_1S_0=11, I will appear at D_3. The following truth table (Fig.48 (b)) summarizes this operation.



Truth table

SELECT Input		Output			
S_1	S_0	D_0	D_1	D_2	D_3
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

(b)

Fig.48: (a) Block diagram of a 1:4 demultiplexer
(b) Its truth table

From the above truth table, the logic expressions for outputs are as follows:

$$D_0 = \overline{S}_1\overline{S}_0 I,\ D_1 = \overline{S}_1 S_0 I,\ D_2 = S_1\overline{S}_0 I,\ and\ D_3 = S_1 S_0 I$$

The logic circuit of the 1:4 demultiplexer is implemented in Fig.49.

CamScanner

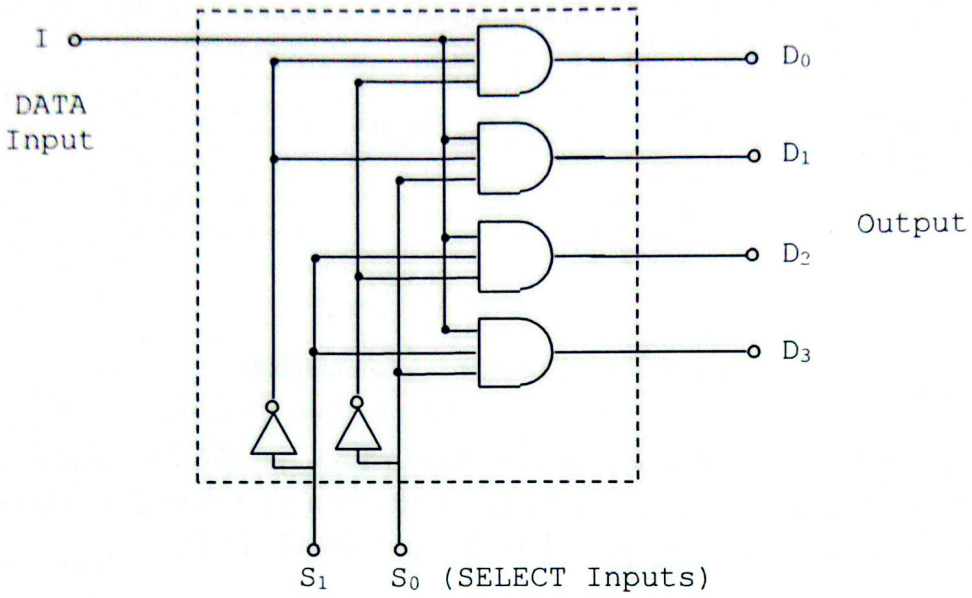
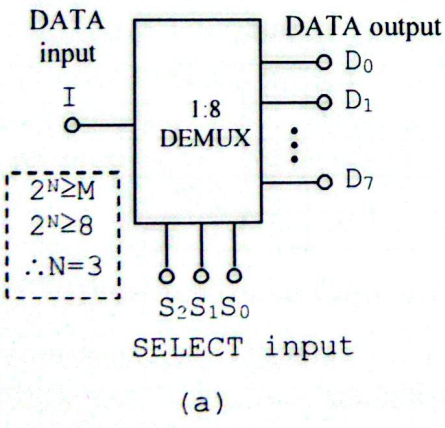


Fig.49: Logic circuit of a 1:4 demultiplexer

7.13.2 1-line-to-8-line (1:8) Demultiplexer

This demultiplexer has a single DATA input line and eight DATA output lines (i.e. M=8). To select these 8 DATA output lines, at least three SELECT lines are required. Fig.50 (a) shows the logic symbol of the 1-line-to-8-line demultiplexer. The SELECT input lines S_2, S_1, and S_0 determine to which output line, the data from input I will be transmitted. The following truth table (Fig.50 (b)) summarizes this operation.



Truth table

SELECT Input			Data Output							
S_2	S_1	S_0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	I	0	0	0	0	0	0	0
0	0	1	0	I	0	0	0	0	0	0
0	1	0	0	0	I	0	0	0	0	0
0	1	1	0	0	0	I	0	0	0	0
1	0	0	0	0	0	0	I	0	0	0
1	0	1	0	0	0	0	0	I	0	0
1	1	0	0	0	0	0	0	0	I	0
1	1	1	0	0	0	0	0	0	0	I

(b)

Fig.50: (a) Block diagram of a 1:8 demultiplexer
(b) Its truth table

From the truth table of Fig.50 (b), we can write the following logic expressions for outputs D_0 through D_7:

$$D_0 = \overline{S}_2\overline{S}_1\overline{S}_0 I, \quad D_1 = \overline{S}_2\overline{S}_1 S_0 I, \quad D_2 = \overline{S}_2 S_1\overline{S}_0 I, \quad D_3 = \overline{S}_2 S_1 S_0 I$$
$$D_4 = S_2\overline{S}_1\overline{S}_0 I, \quad D_5 = S_2\overline{S}_1 S_0 I, \quad D_6 = S_2 S_1\overline{S}_0 I, \quad D_7 = S_2 S_1 S_0 I$$

The logic circuit of the 1:8 demultiplexer from the above logic expressions is implemented in Fig.51.

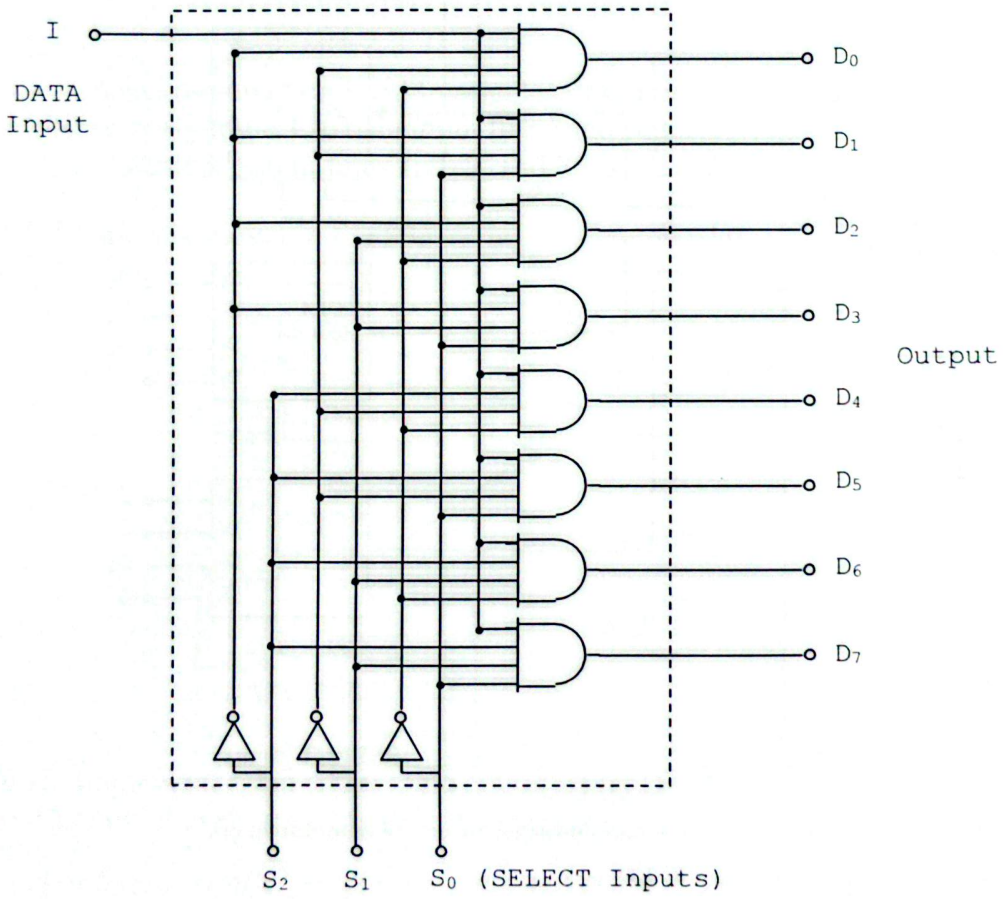


Fig.51: Logic circuit of a 1:8 demultiplexer

7.14 Larger DEMUXs

Larger demultiplexers can be constructed using smaller ones. For example, a 1:16 demultiplexer can be constructed using 1:4 demultiplexers.

CamScanner

1:16 DEMUX using 1:4 DEMUXs: The implementation of a 1:16 DEMUX using 1:4 DEMUXs is shown below in Fig.52. A 1:16 DEMUX has 16 data output lines. Thus, to distribute data over these data output lines, at least four select lines ($S_3S_2S_1S_0$) are required.

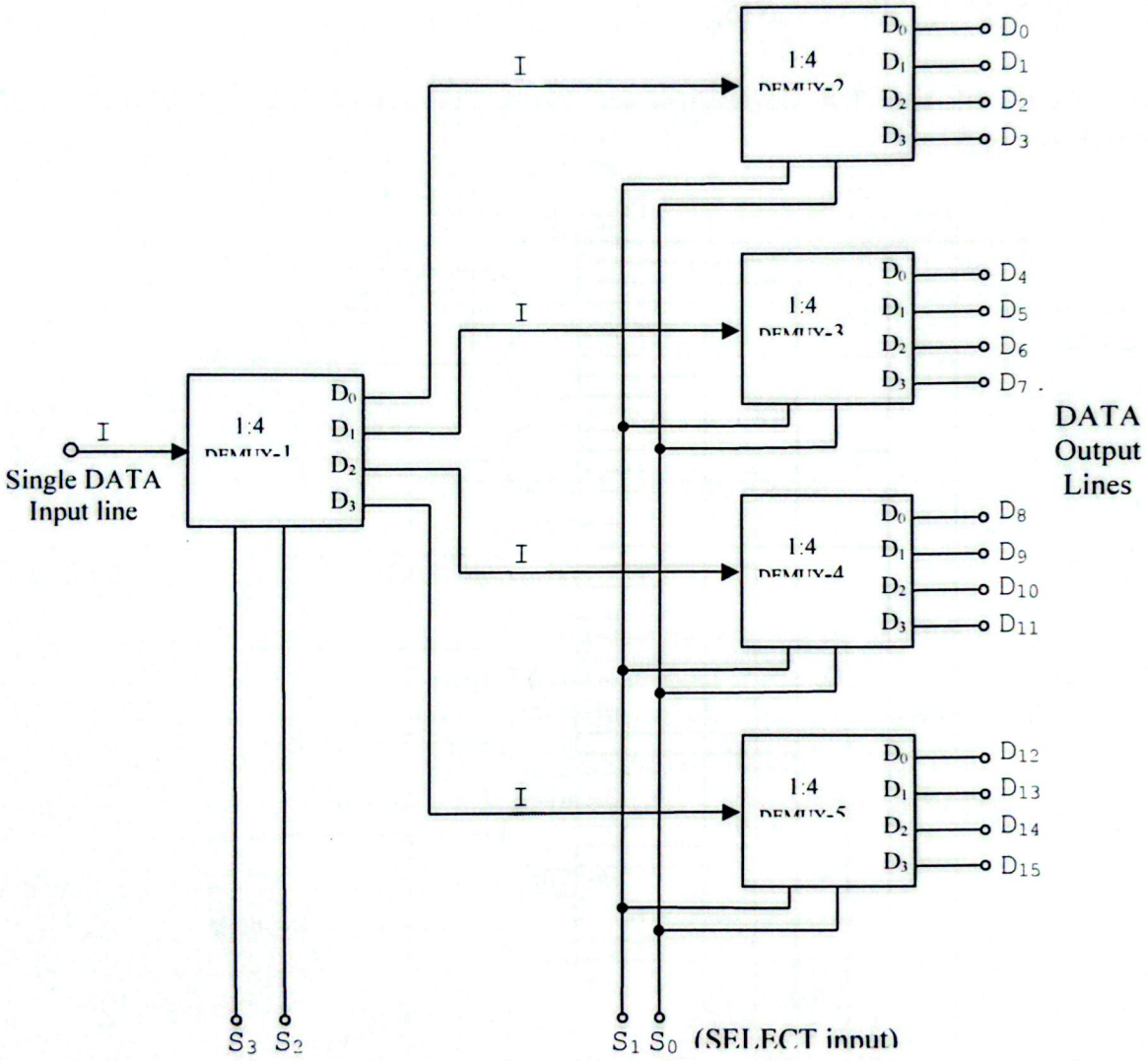


Fig.52: A 1:16 demultiplexer using 1:4 demultiplexers

The operation of this logic circuit can be explained with an example. Suppose, the select input $S_3S_2S_1S_0$=1011 is applied. Thus, S_3S_2=10 will select D_2 output of DEMUX-1 so that, data input I can pass onto DEMUX-4. Now since S_1S_0=11 will select D_3 output of DEMUX-4, data input I will appear at the output line D_{11}.

CamScanner

7.15 Boolean Function Implementation using DEMUX

Demultiplexers can be used to implement any Boolean function of n-variables. However, in that case, the data input line I is to be connected to logic 1 (HIGH).

Example 11. Implement the function $Y(A,B,C) = \overline{A}B + BC + \overline{A}C$ using a DEMUX.

Solution: i) Express the function in standard SOP form.

$$Y(A,B,C) = \overline{A}B + BC + \overline{A}C$$
$$= \overline{A}BC + \overline{A}B\overline{C} + ABC + \overline{A}\,\overline{B}C$$
$$= \sum m(1,2,3,7)$$

Since the function is of 3 variables, it requires a 1:8 DEMUX.

ii) Now apply logic 1 to data input line I and finally,

iii) OR the data output lines 1, 2, 3, and 7.

The implementation is shown in Fig.53.

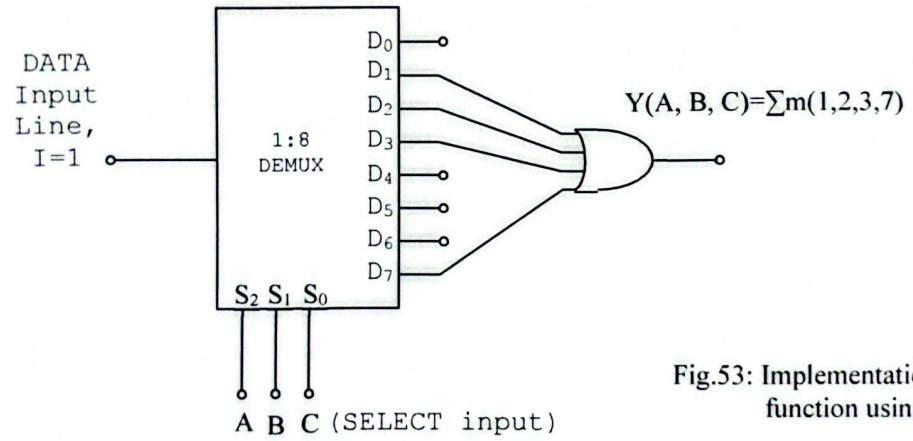


Fig.53: Implementation of logic function using DEMUX

Example 12. Implement a full adder logic circuit using DEMUXs.

Solution: The output expressions for a full adder are

$$For\ Sum, S = \overline{A}B\overline{C_i} + A\overline{B}\,\overline{C_i} + \overline{A}\,\overline{B}C_i + ABC_i = \sum m(1,2,4,7)$$

$$For\ Carry, C_o = AB\overline{C_i} + \overline{A}BC_i + ABC_i + A\overline{B}C_i = \sum m(3,5,6,7)$$

The expressions are already in standard SOP form. Since there are 3 input variables (A, B, C_i), it requires a 1:8 DEMUX for implementation. Now apply logic 1 to data input line I and finally,

i) OR the data output lines 1, 2, 4, and 7 for S and

ii) OR the data output lines 3, 5, 6, and 7 for C_o.

CamScanner

The implementation is shown in Fig.54.

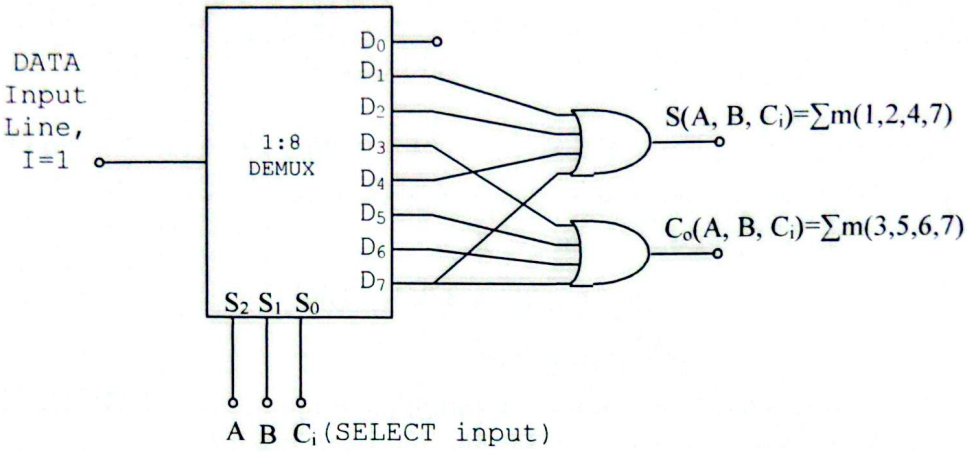


Fig.54: Implementation of full adder using DEMUX

7.16 Demultiplexer as a Decoder

A demultiplexer can be used as a decoder, if its data input line I is always true (i.e. logic 1). In that case, the input code (i.e. the set of inputs) is applied to the select inputs of the DEMUX. This activates only one of several outputs corresponding to that input code. As an example, a binary-to-octal decoder has been implemented in Fig.55.

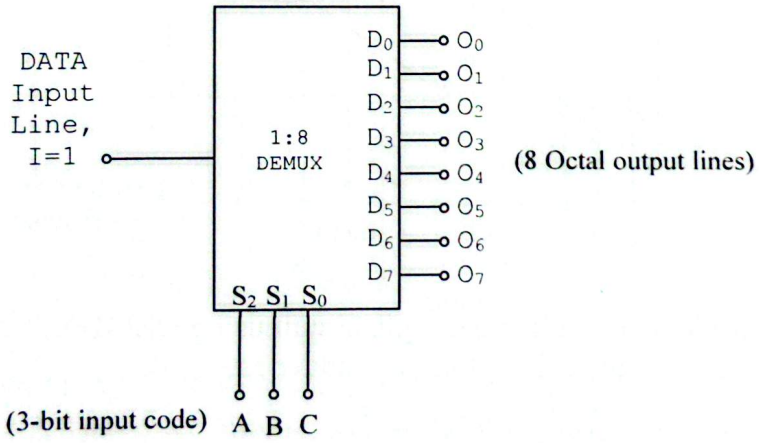


Fig.55: Implementation of binary-to-octal decoder using DEMUX

Suppose that a input code ABC=001 is applied, this will activates the output line O_1, thus indicating that this is Octal digit 1. Similarly, if we input a code 101, the output line O_5 will be activated indicating Octal digit 5. Clearly, the DEMUX implementation of Fig.55 performs as a binary-to-octal decoder.

7.17 Applications of Demultiplexers

CamScanner

Demultiplexers have found a wide variety of applications in digital systems. Some of the major applications of demultiplexers are serial-to-parallel data conversion, security monitoring systems, synchronous data transmission systems, etc.

REVIEW QUESTIONS

1. What is an encoder? Design a decimal-to-BCD encoder.
2. Design an Octal-to-Binary / Decimal-to-Excess-3 encoder.
3. How does a priority encoder differ from an ordinary encoder?
4. What is a priority encoder? Design a decimal-to-BCD priority encoder.
5. Design an octal-to-binary priority encoder.
6. Design a 4-line-to-2-line priority encoder.
7. Mention some applications of encoders.
8. What is a decoder? How does a decoder differ from an encoder?
9. Design a 3-to-8-line decoder/BCD-to-Decimal decoder.
10. Design a 4-line-to-16-line decoder.
11. Draw and explain the logic circuit of a BCD-to-7-segment decoder.
12. Discuss the basic operating principle of Liquid Crystal Display.
13. Design a 4-line-to-16-line decoder using 3-line-to-8-line decoder.
14. Design a 5-line-to-32-line decoder using 3-line-to-8-line decoders.
15. Design a full-adder logic circuit using decoders.
16. Design a full-subtractor logic circuit using decoders.
17. Implement the function $Y = \overline{A}B + BC + \overline{A}C$ using a decoder.
18. Explain how a decoder can be used as a demultiplexer.
19. A logic circuit is defined by the following functions:

    $f_1 = \overline{AB} + AB\overline{C}$

    $f_2 = \overline{A} + B$

    $f_3 = AB + \overline{AB}$

    Draw the circuit with a decoder and external gates.
20. A combinational circuit is defined by the following two functions:
    i. $F_1(A,B)=\sum m(0,3)$
    ii. $F_2(A,B)=\sum m(1\text{-}3)$

    Implement the circuit using a decoder and external gates.

21. Implement the following functions using a decoder:

CamScanner

$F_1=\sum m(1,2,4,7,8,11\text{-}13)$
$F_2=\sum m(2,3,9,11)$
$F_3=\sum m(10,12\text{-}14)$
$F_4=\sum m(2,4,8)$

22. Mention some applications of decoders.
23. What is a multiplexer? Explain the basic operation of a multiplexer.
24. Design a 4-to-1-line multiplexer.
25. Construct a 4-to-1-line multiplexer using 2-to-4-line decoder.
26. Implement an 8:1 multiplexer using 4:1 multiplexers ICs.
27. Design a 32:1 multiplexer using 16:1 multiplexer ICs.
28. Design a 32:1 multiplexer using 8:1 and 4:1 multiplexers ICs.
29. Design a 40:1 multiplexer using 8:1 multiplexers.
30. Realize the following function using a multiplexer:
    $F(A,B,C,D)=\sum m(0,1,3,4,8,9,15)$
31. Realize the following functions using a MUX:
    i. $f(A,B,C)=\sum m(1,3,5,6)$
    ii. $f(A,B,C,D)=\sum m(0,1,3,4,8,9,15)$
    iii. $f(A,B,C,D)=\sum m(0,2,3,6,8,9,12,14)$
32. Realize the following functions using a MUX:
    $f_1 = \overline{AB} + AB\overline{C}$
    $f_2 = \overline{A} + B$
    $f_3 = AB + \overline{AB}$
33. Realize the function
    $F(A,B,C,D)=\sum m(2,4,6,7,9\text{-}12,15)$
    Using i) a 16:1 multiplexer and ii) an 8:1 multiplexer.
34. Implement a full-adder logic circuit using a multiplexer.
35. Design a BCD-to-Gray code converter using MUXs.
36. Explain how a multiplexer can be used to convert parallel data to serial form.
37. What are some of the major applications of MUXs?
38. What is a demultiplexer? Explain the basic operation of a demultiplexer.
39. Explain the difference between a MUX and a DEMUX.
40. Design a 1-to-8-line demultiplexer.
41. Draw and explain the logic circuit of a 1:4/1:8 demultiplexer.
42. Design a 1:16 demultiplexer using 1:4 demultiplexers ICs.
43. Design a 1:40 demultiplexer using 1:8 demultiplexers ICs.
44. Implement the function $Y = \overline{AB} + BC + \overline{AC}$ using a demultiplexer.
45. Implement a full-adder logic circuit using a demultiplexer.
46. Explain how a demultiplexer can be used as a decoder.
47. Mention some applications of demultiplexers.

CamScanner